

AI VIET NAM

@aivietnam.edu.vn

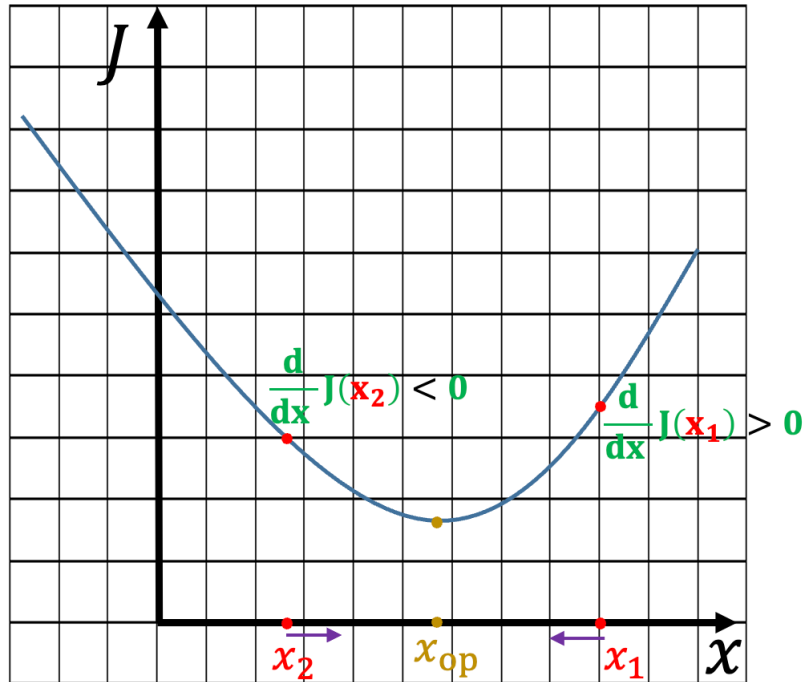
Linear Regression

An Optimization Approach

Quang-Vinh Dinh
PhD in Computer Science

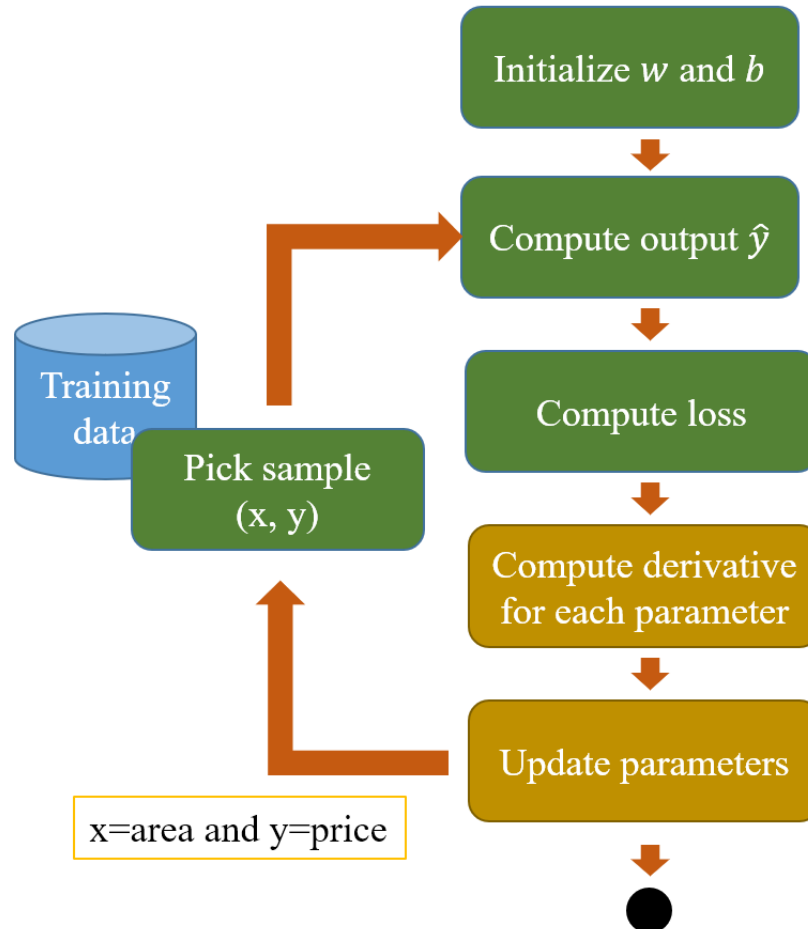
Objectives

Gradient descent



$$\mathbf{x}_{new} = \mathbf{x}_{old} - \eta \frac{d}{dx} J(\mathbf{x}_{old})$$

Linear Regression

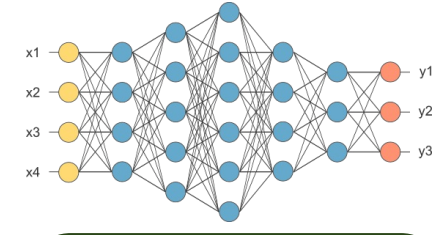
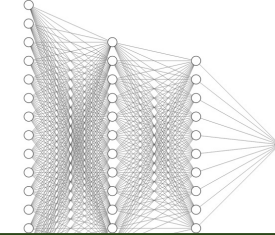
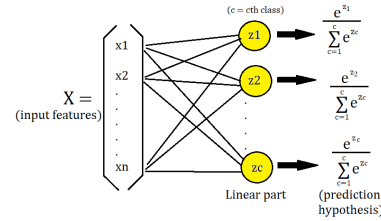
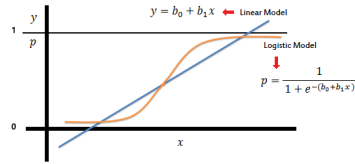


Vectorization

$$\begin{cases} b = b - \eta \frac{\partial L}{\partial b} \\ w = w - \eta \frac{\partial L}{\partial w} \\ \theta = \theta - \eta \nabla_{\theta} L \end{cases}$$

$$\rightarrow \theta = \theta - \eta \nabla_{\theta} L$$

Motivation



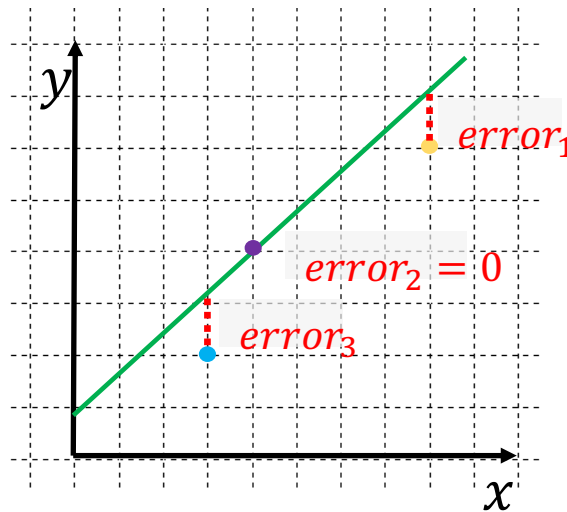
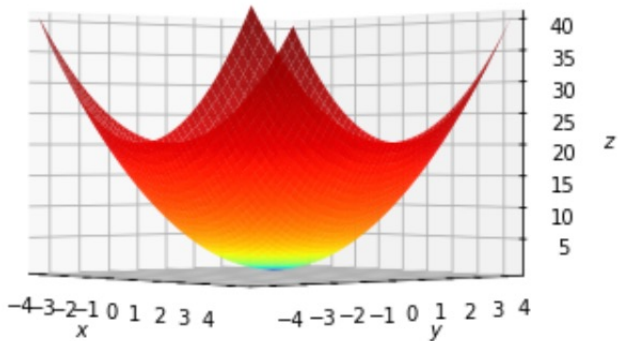
Linear
Regression

Logistic
Regression

Softmax
Regression

Multilayer
Perceptron

Deep
Learning



- 1) Pick a sample $(\mathbf{x}, y)$ from training data
- 2) Compute output $\hat{y}$
$$\hat{y} = \boldsymbol{\theta}^T \mathbf{x} = \mathbf{x}^T \boldsymbol{\theta}$$
- 3) Compute loss
$$L = (\hat{y} - y)^2$$
- 4) Compute derivative
$$\nabla_{\boldsymbol{\theta}} L = 2\mathbf{x}(\hat{y} - y)$$
- 5) Update parameters
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

Outline

SECTION 1

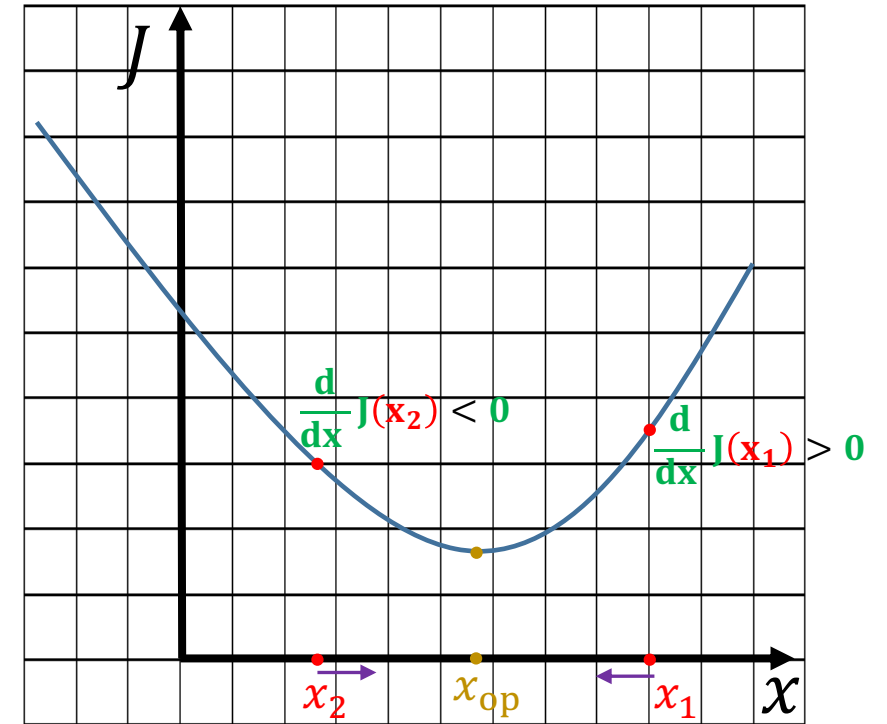
Gradient descent

SECTION 2

Linear Regression

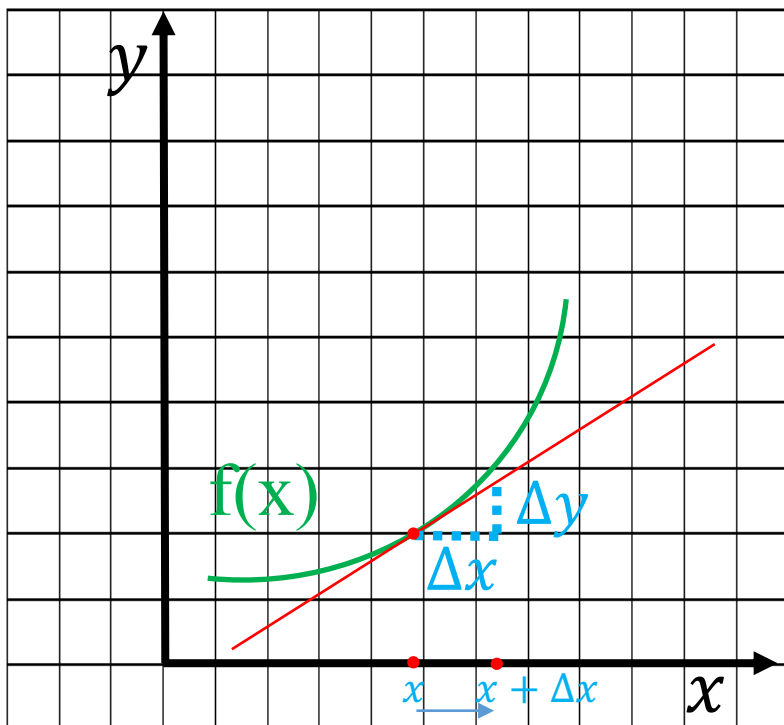
SECTION 3

Vectorization



$$\mathbf{x}_{new} = \mathbf{x}_{old} - \eta \frac{d}{dx} J(\mathbf{x}_{old})$$

❖ Đạo hàm cho hàm liên tục



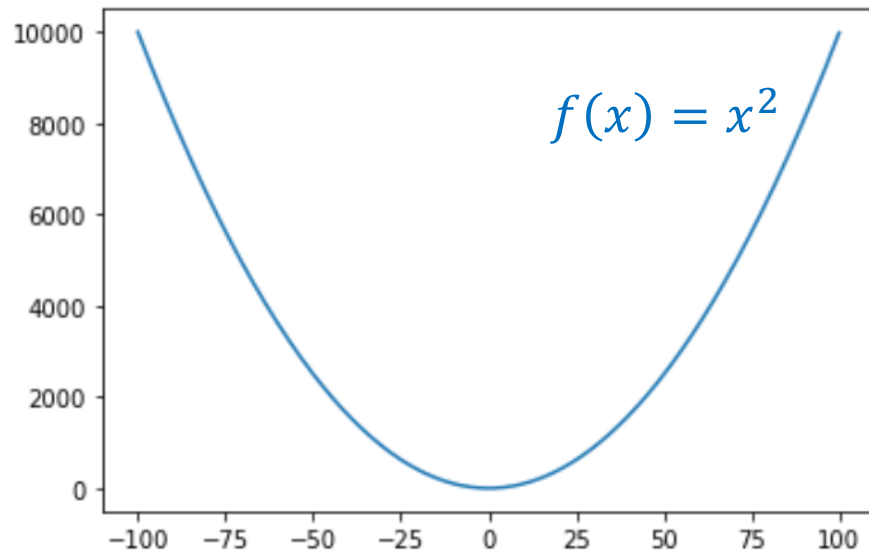
$$\frac{d}{dx} f(x), \frac{dy}{dx}, y', f'(x)$$

$$\text{Đạo hàm} = \frac{\text{Thay đổi theo } y}{\text{Thay đổi theo } x} = \frac{\Delta y}{\Delta x}$$

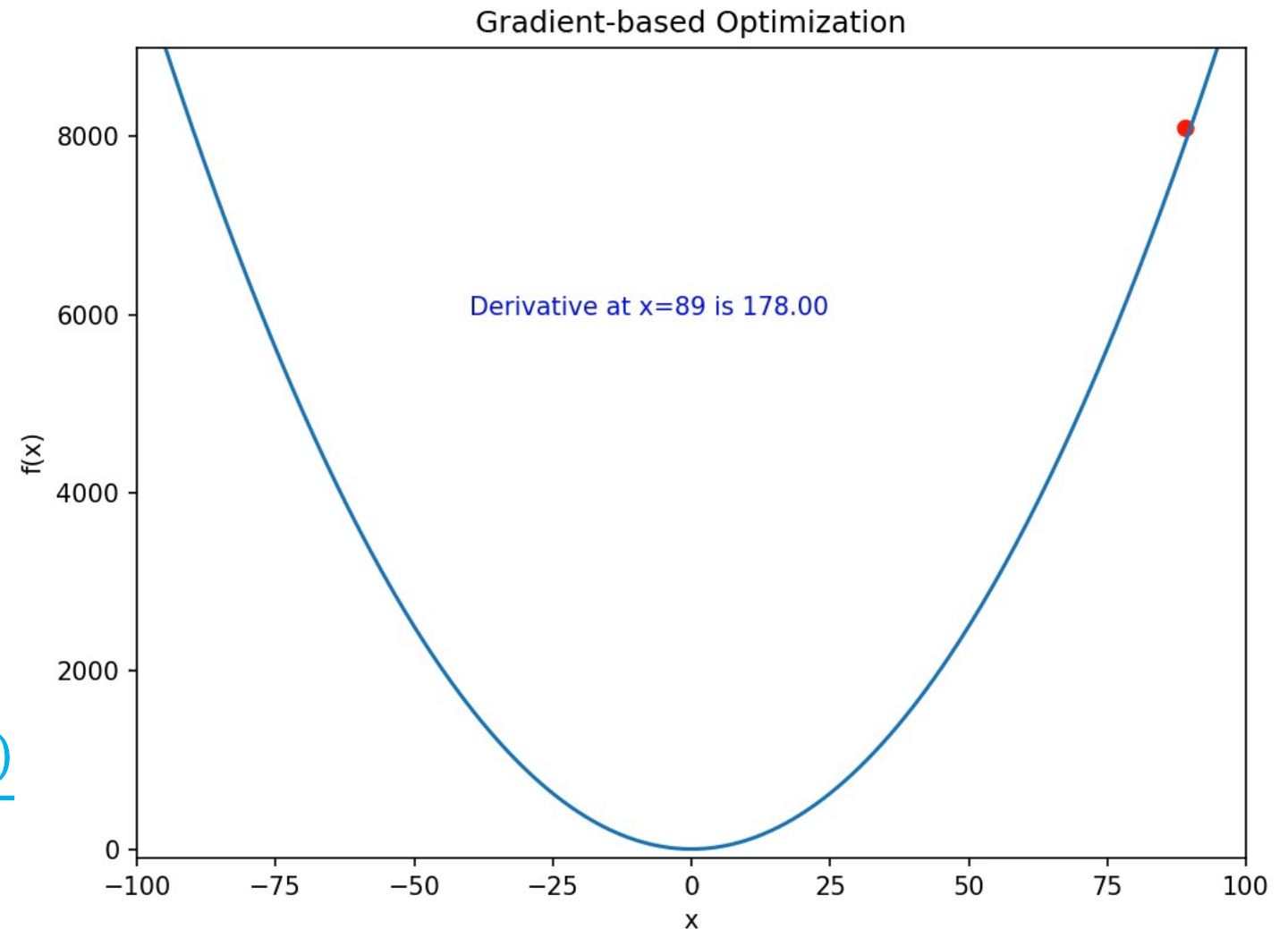
$$\frac{d}{dx} f(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Δx cần tiến về 0 để
đường tiếp tuyến tiến
về hàm $f(x)$ trong vùng
lân cận tại x

❖ Square function



$$\frac{d}{dx} f(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

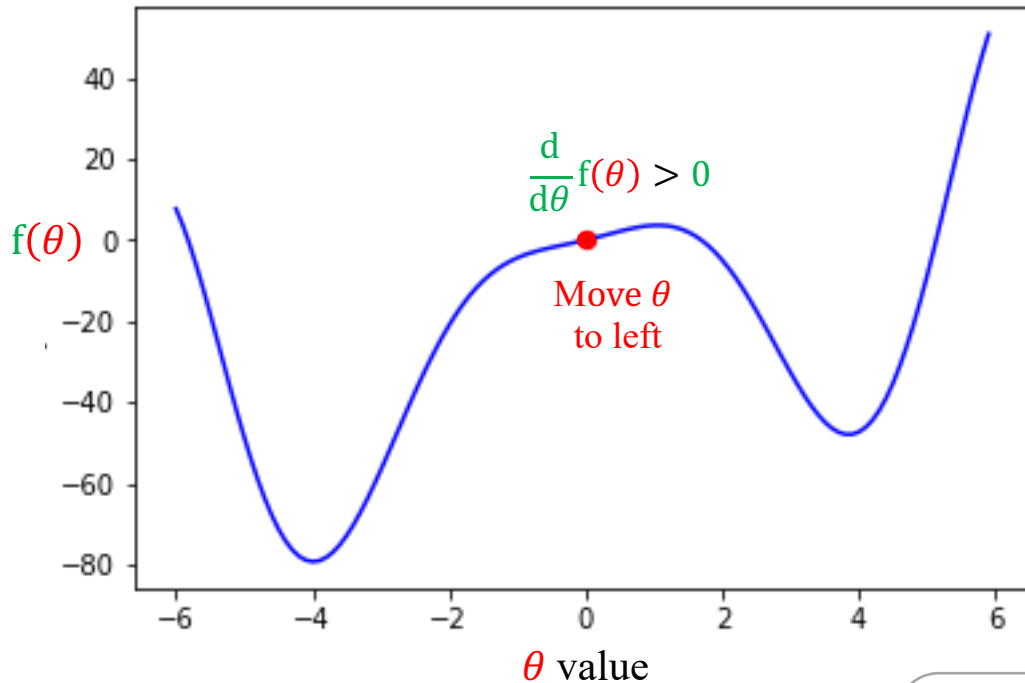


Derivative and Optimization

❖ Test an idea

❖ Moving opposite of derivative direction

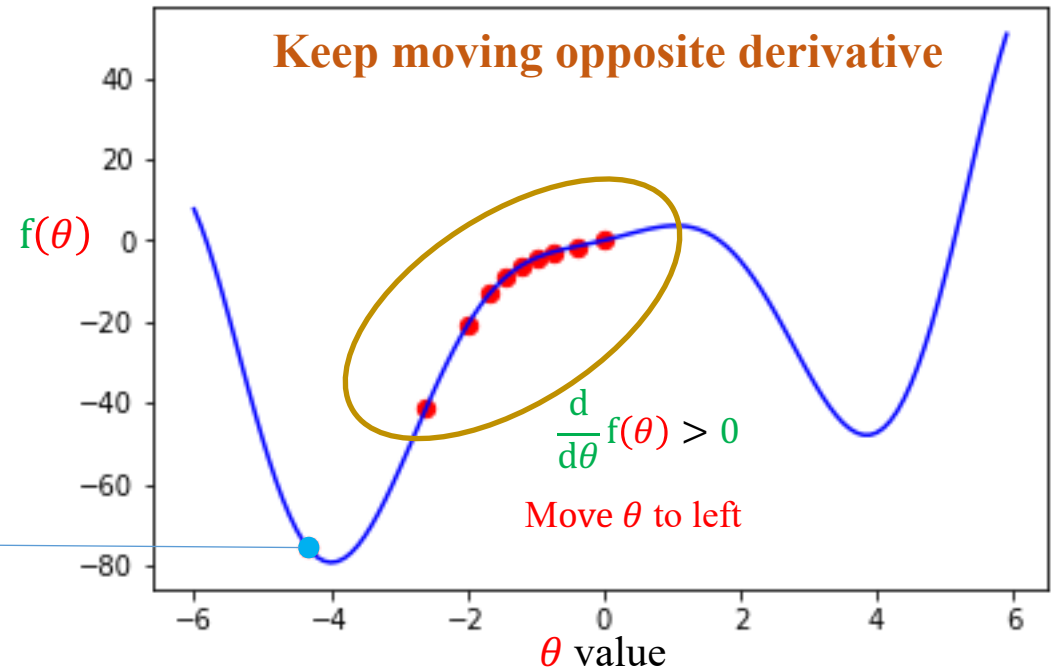
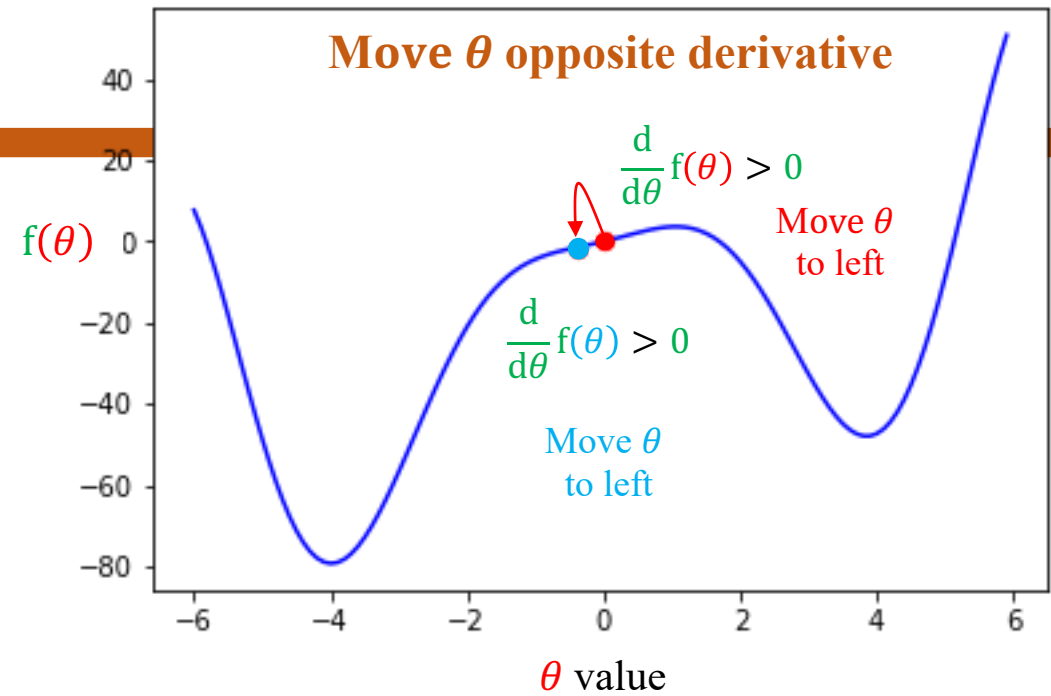
Initialize θ



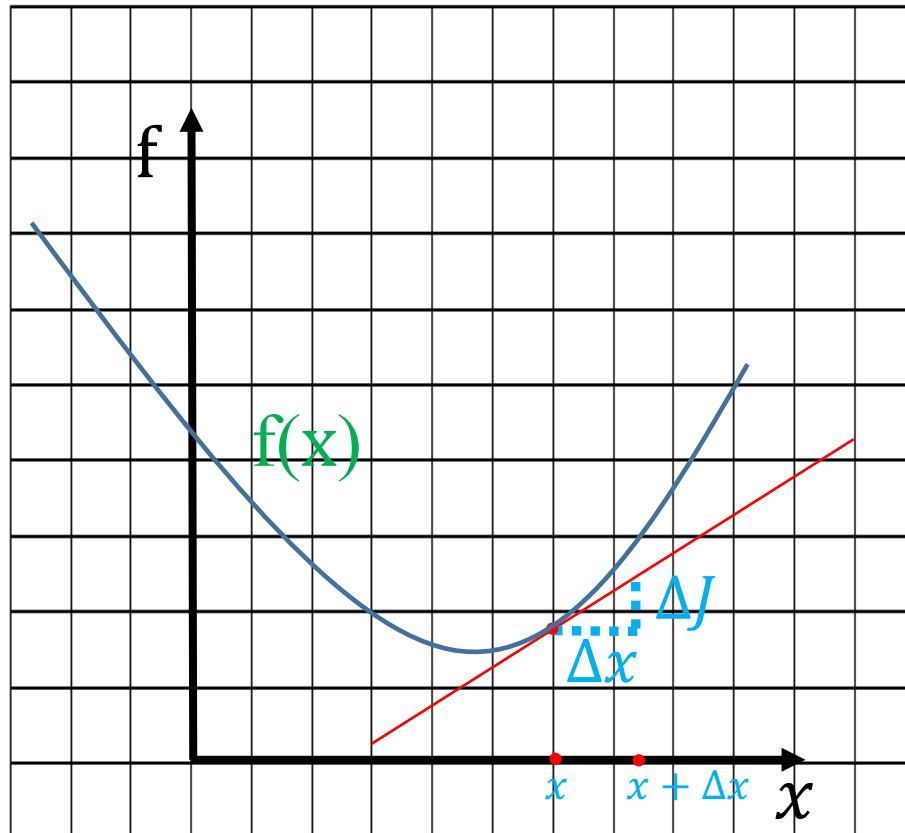
$$\frac{d}{d\theta}f(\theta) < 0$$

Move θ
to right

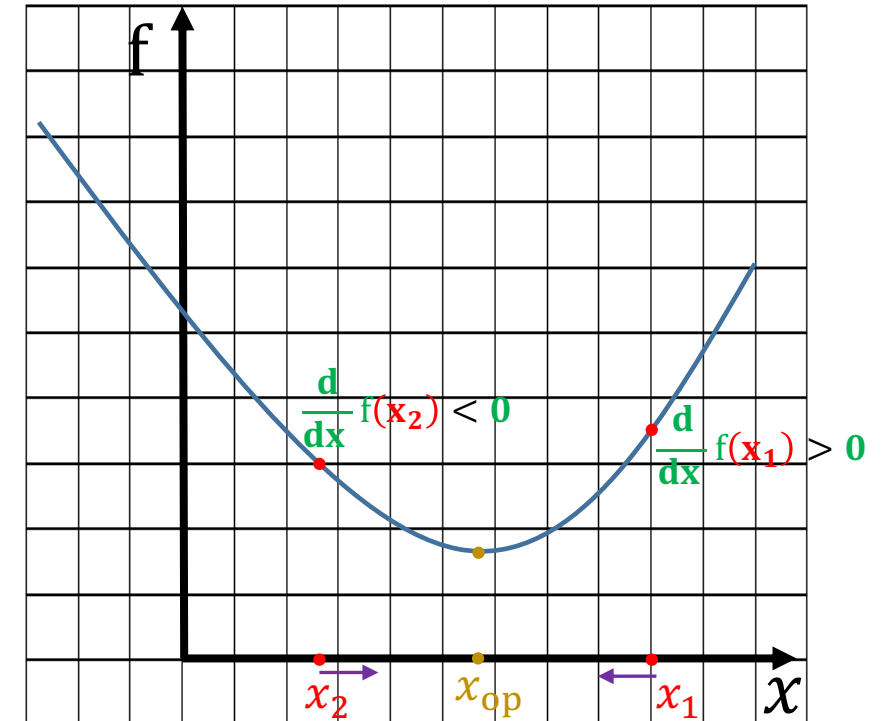
Move θ opposite derivative



❖ Gradient descent



$$\frac{d}{dx} f(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$



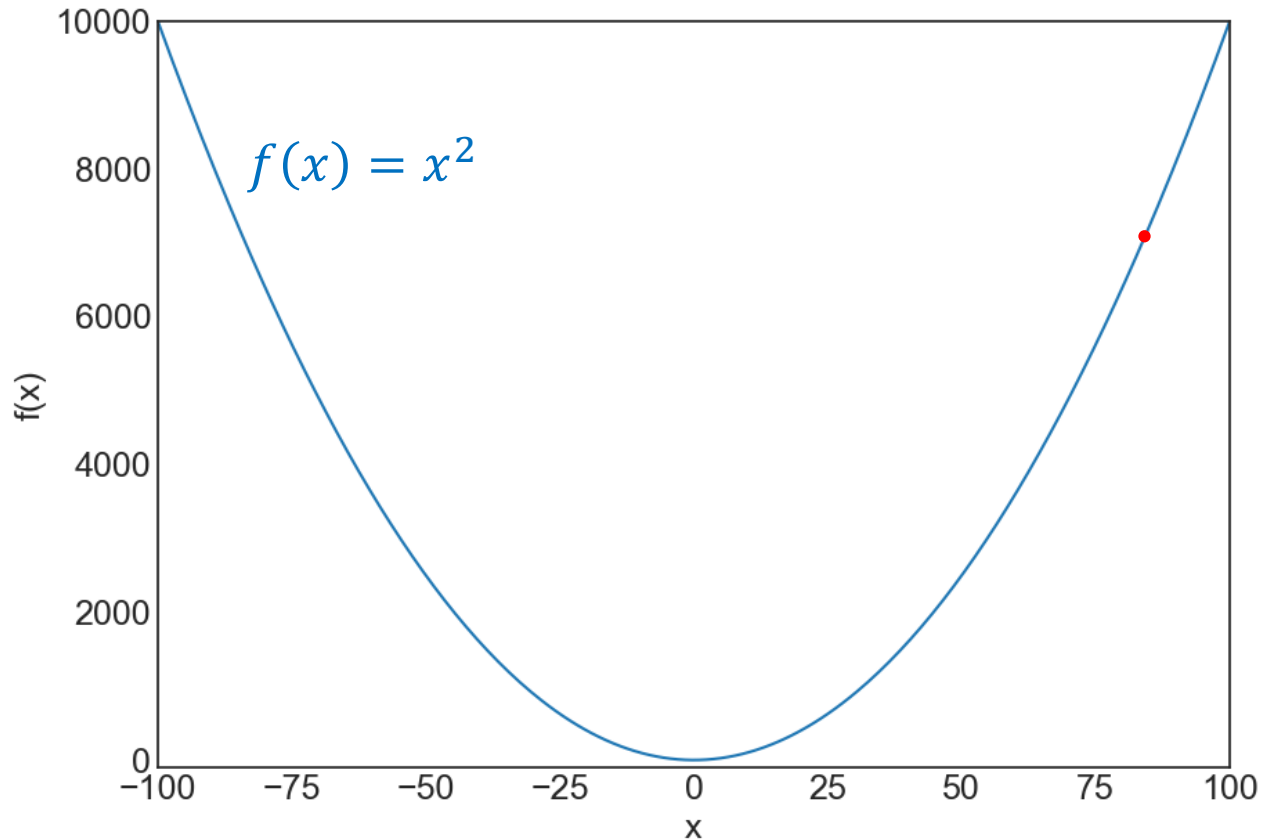
$$x_{new} = x_{old} - \eta \frac{d}{dx} f(x_{old})$$

learning rate

Derivative at x_{old}

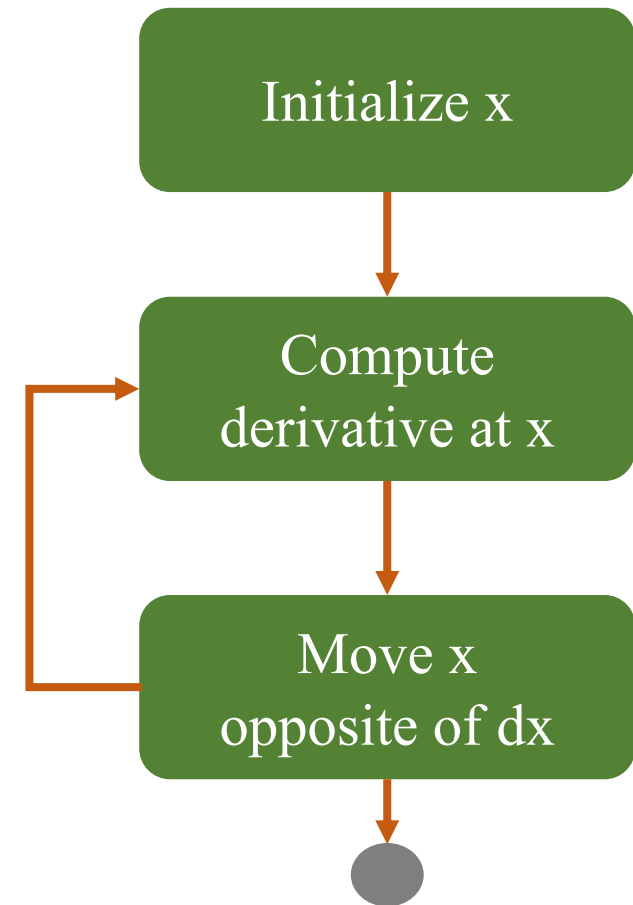
Observation 1

❖ Square function



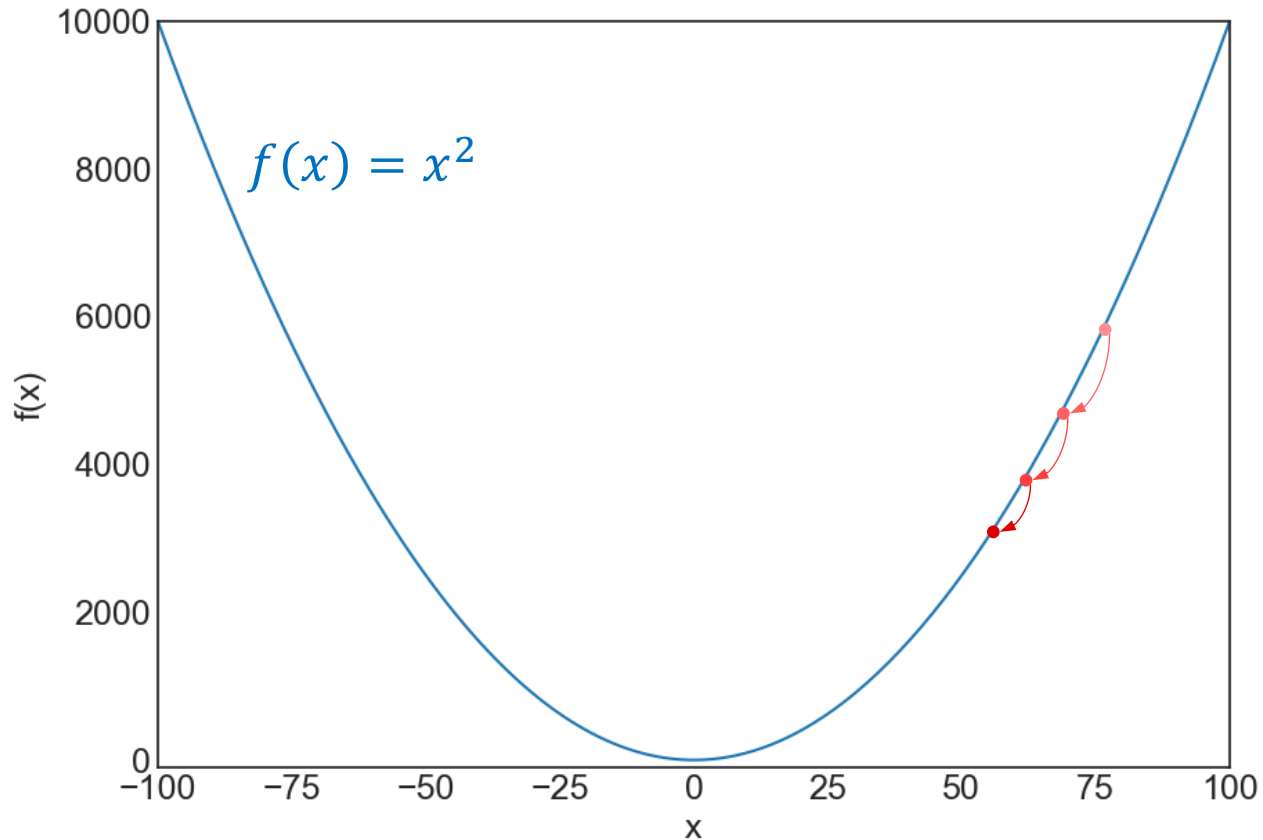
$$-100 \leq x \leq 100$$
$$x \in \mathbb{R}$$

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$



Optimization

❖ Square function



$$\begin{aligned} -100 \leq x \leq 100 \\ x \in \mathbb{R} \end{aligned}$$

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

$$x_0 = 70.0 \quad \eta = 0.1$$

$$f'(x_0) = 140.0$$

$$x_1 = x_0 - \eta f'(x_0) = 56.0$$

$$f'(x_1) = 112.0$$

$$x_2 = x_1 - \eta f'(x_1) = 44.8$$

$$f'(x_2) = 89.6$$

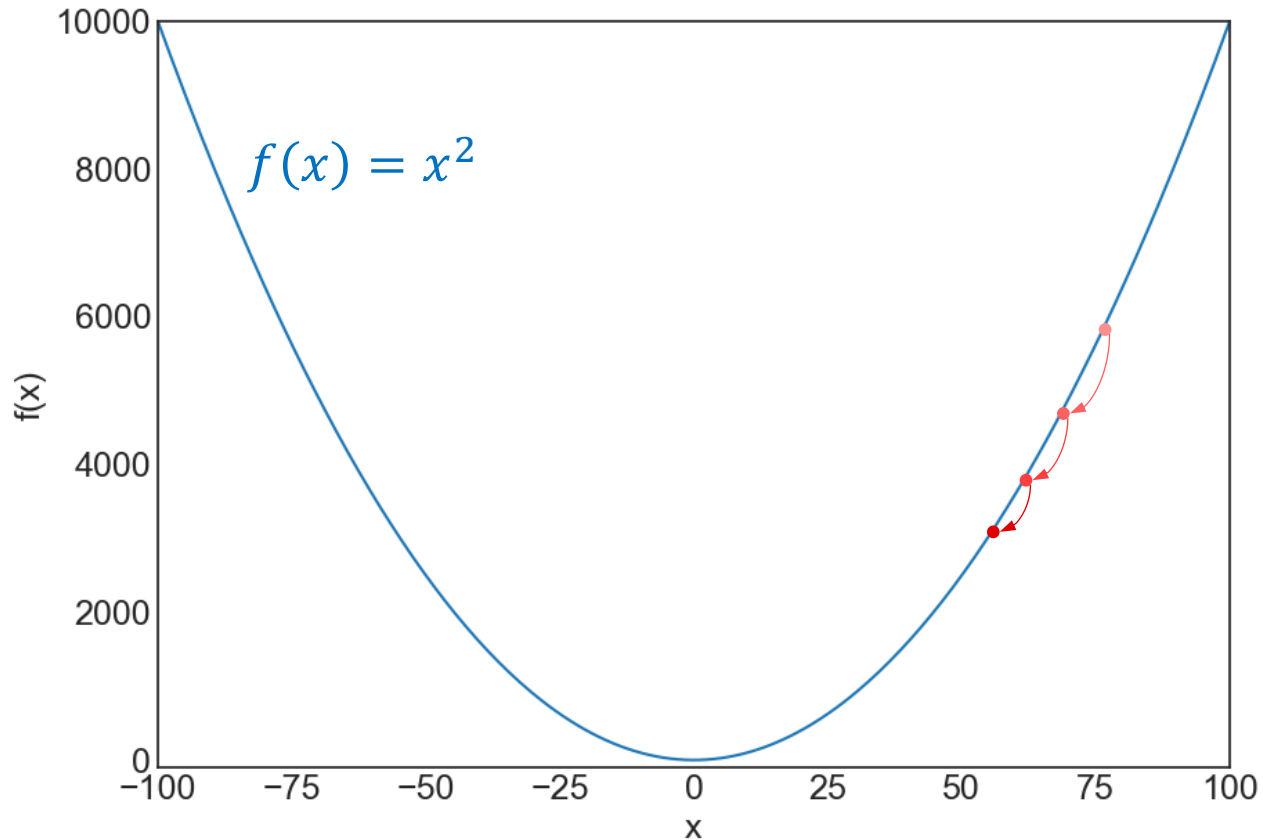
$$x_3 = x_2 - \eta f'(x_2) = 35.84$$

$$f'(x_3) = 71.68$$

$$x_4 = x_3 - \eta f'(x_3) = 28.672$$

Optimization

❖ Square function



Keep doing

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

$$x_{30} = 0.069 \quad \eta = 0.1$$

$$f'(x_{30}) = 0.138$$

$$x_{31} = x_{30} - \eta f'(x_{30}) = 0.055$$

$$f'(x_{31}) = 0.11$$

$$x_{32} = x_{31} - \eta f'(x_{31}) = 0.044$$

$$f'(x_{32}) = 0.088$$

$$x_{33} = x_{32} - \eta f'(x_{32}) = 0.035$$

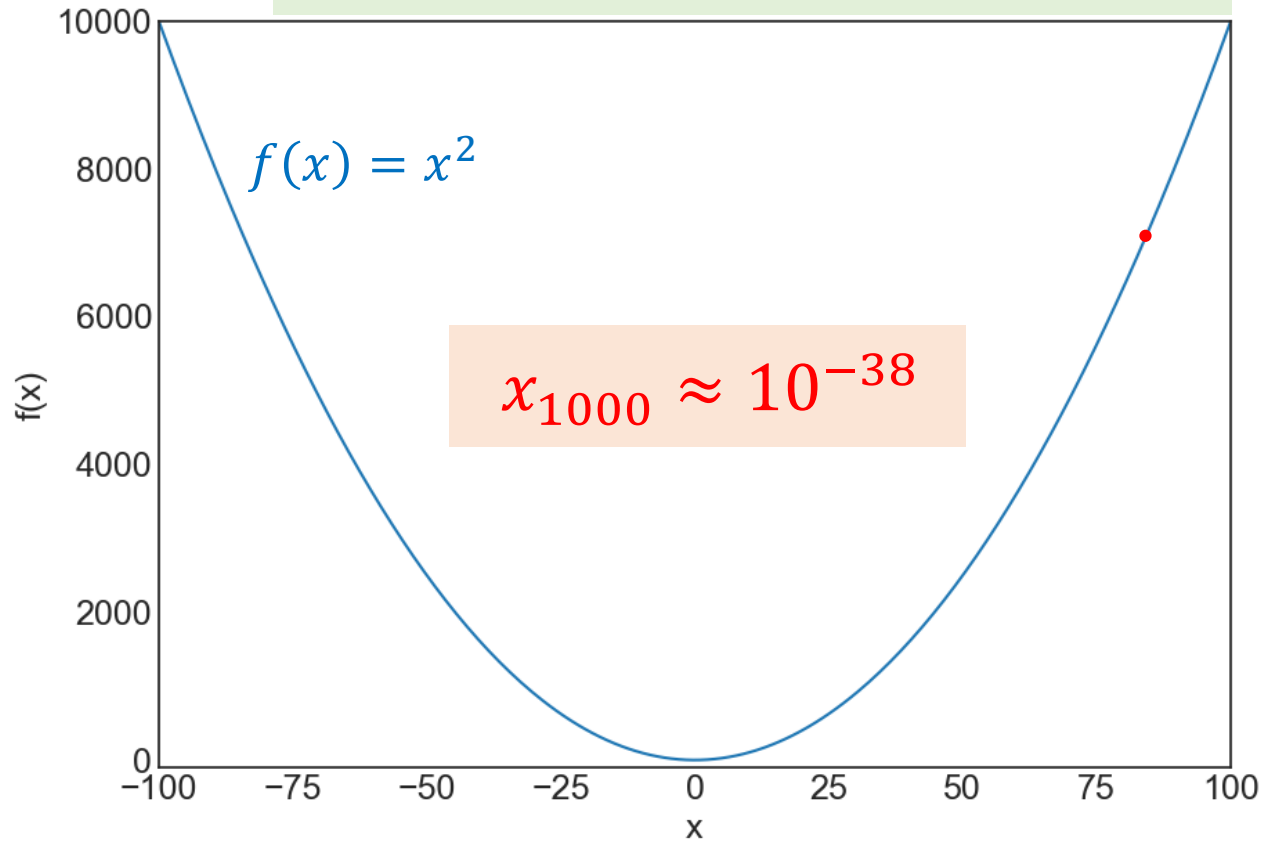
$$f'(x_{34}) = 0.071$$

$$x_{34} = x_{33} - \eta f'(x_{33}) = 0.028$$

Optimization

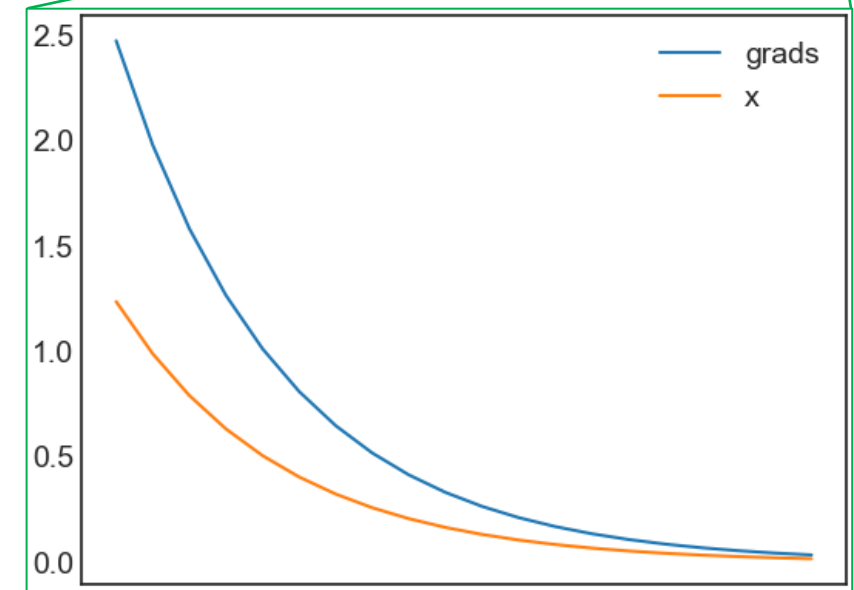
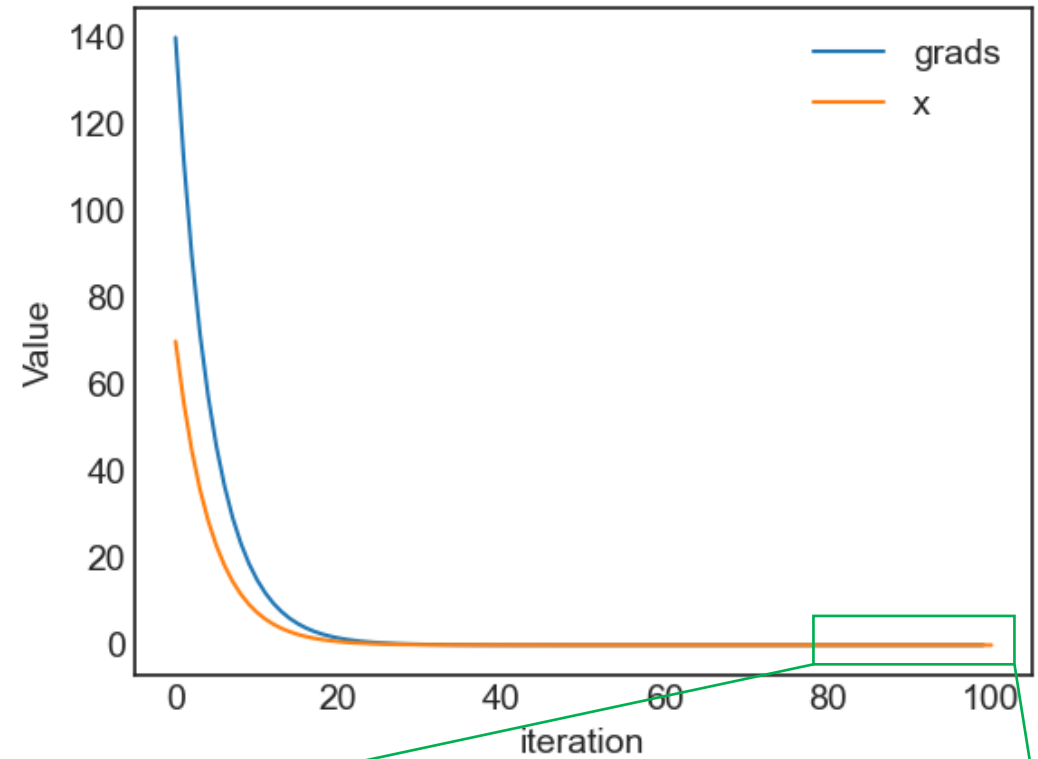
❖ Square function

$$x_t = x_{t-1} - \eta f'(x)$$



$$x_{1000} \approx 10^{-38}$$

Optimized successfully!



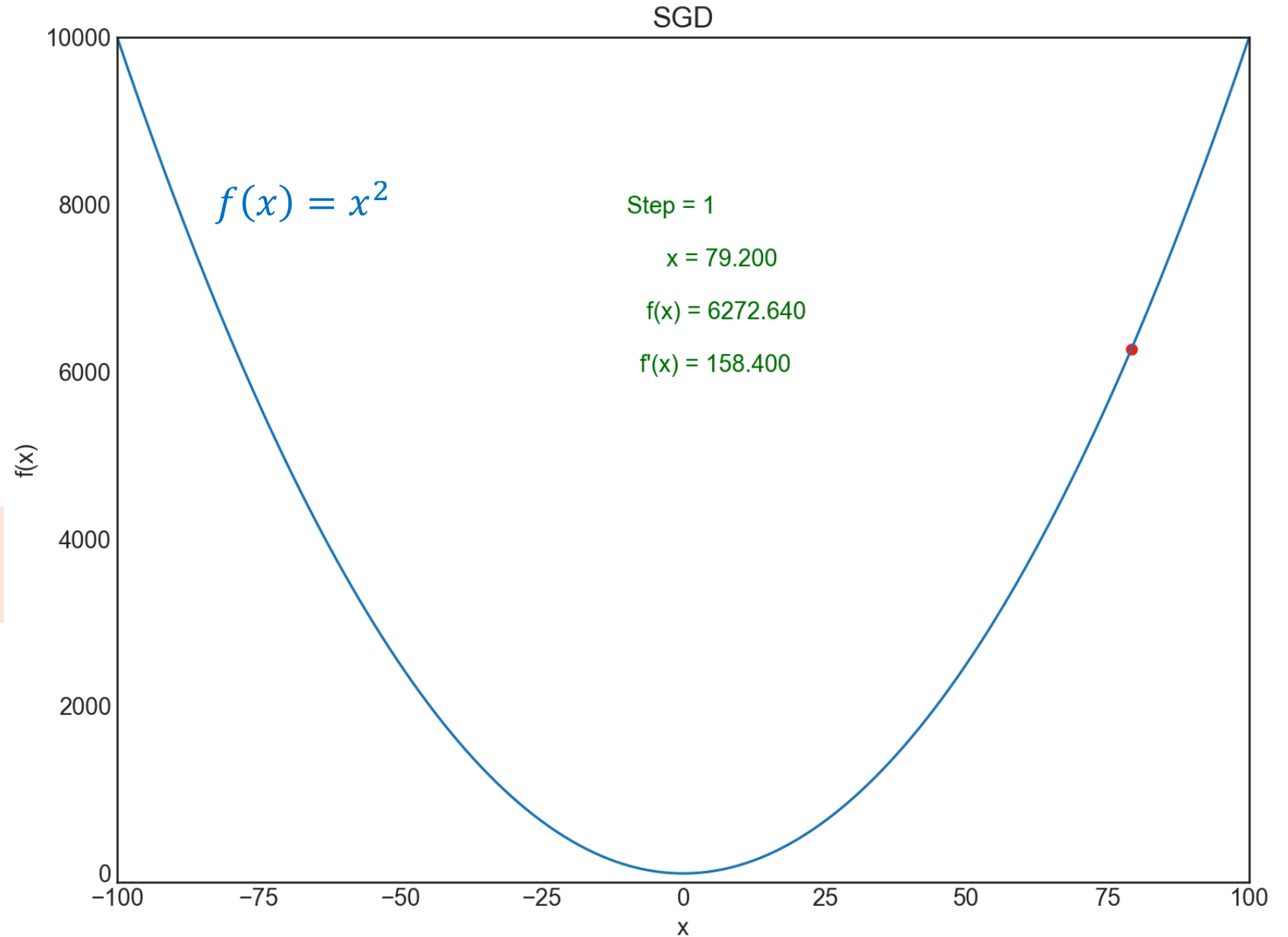
Optimization

❖ Square function

$$x_0 = 99.0$$

$$\eta = 0.1$$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

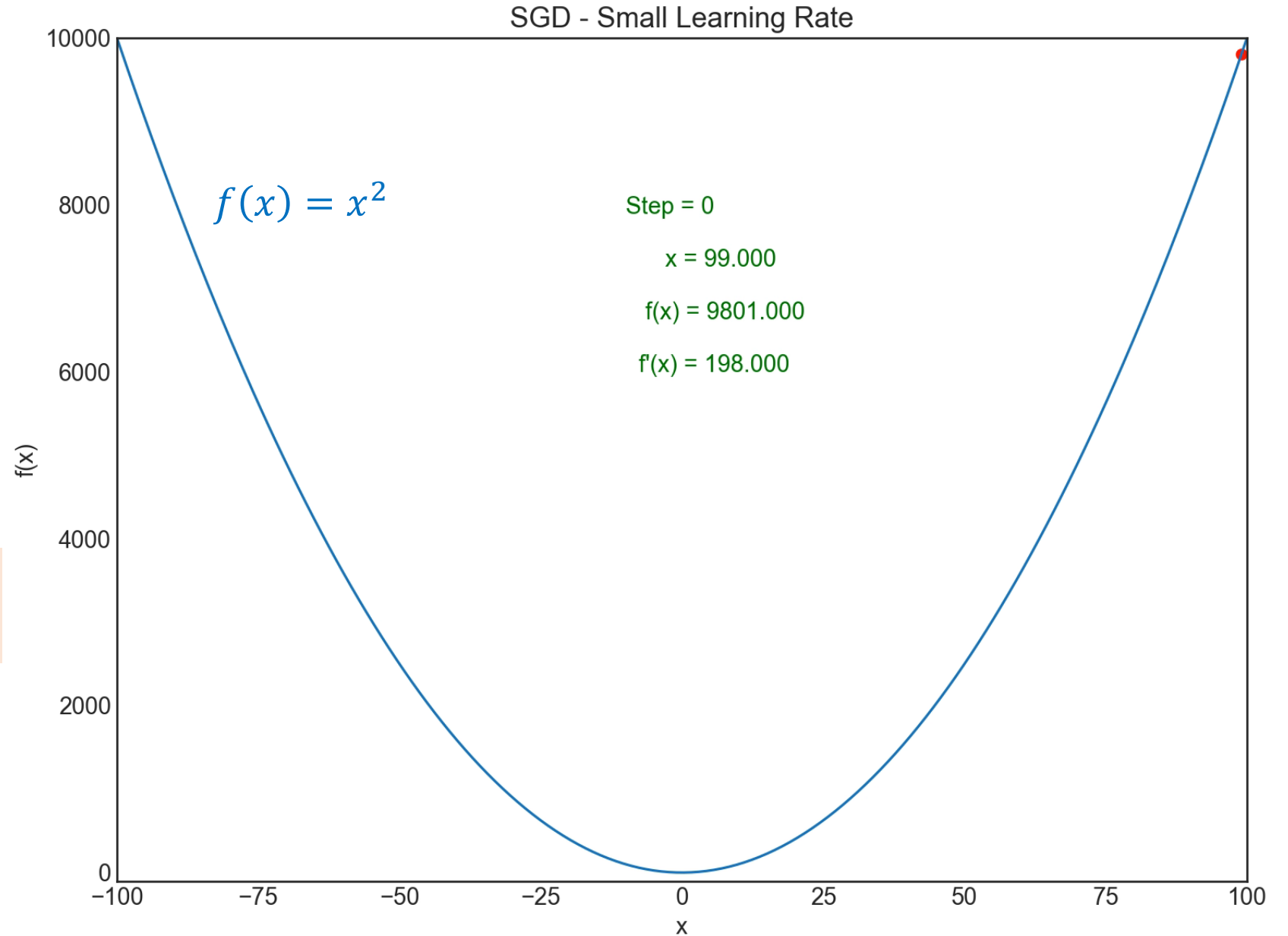
❖ Square function

Discussion

$$x_0 = 99.0$$

$$\eta = 0.001$$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

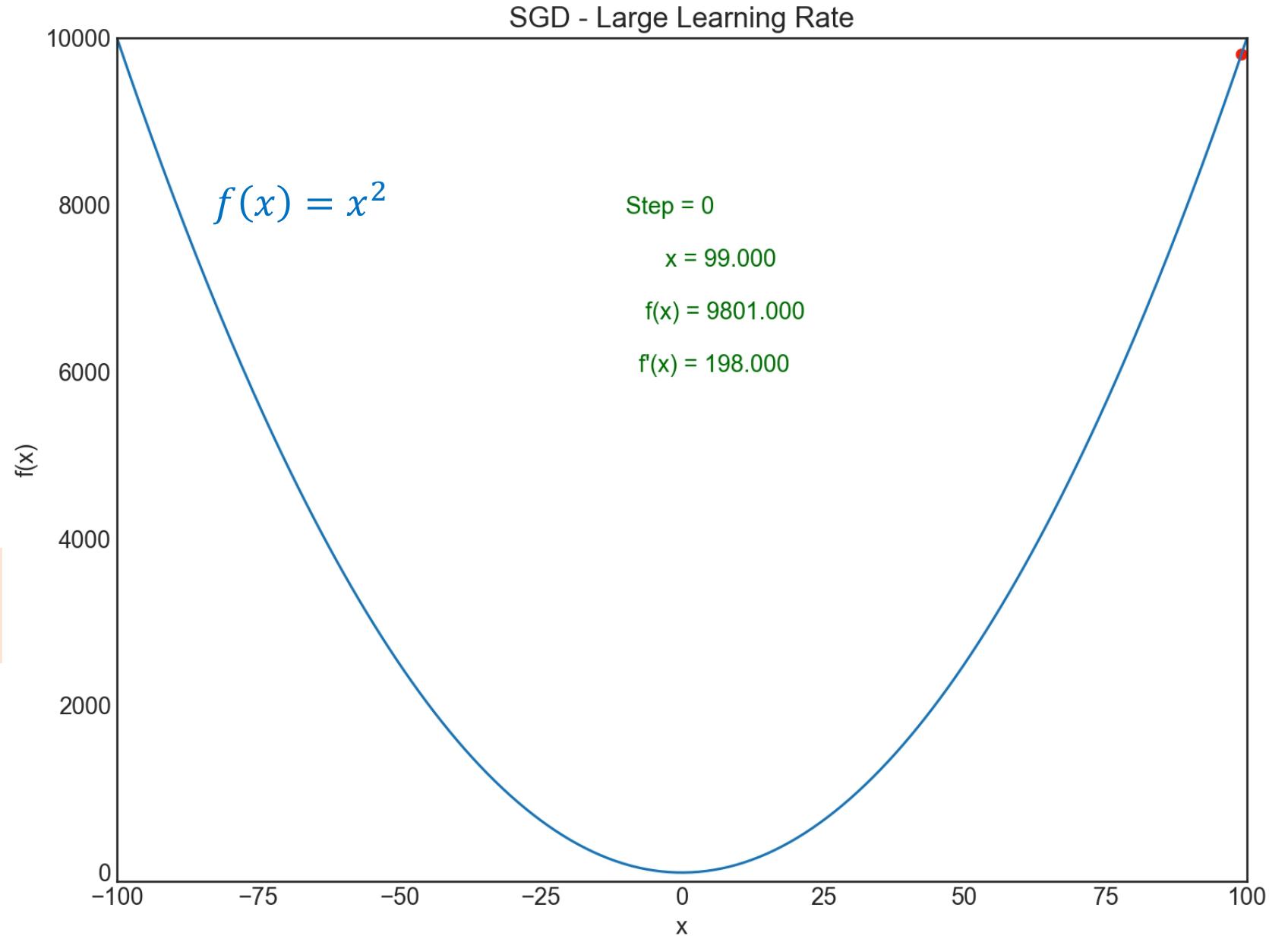
❖ Square function

Discussion

$$x_0 = 99.0$$

$$\eta = 0.8$$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

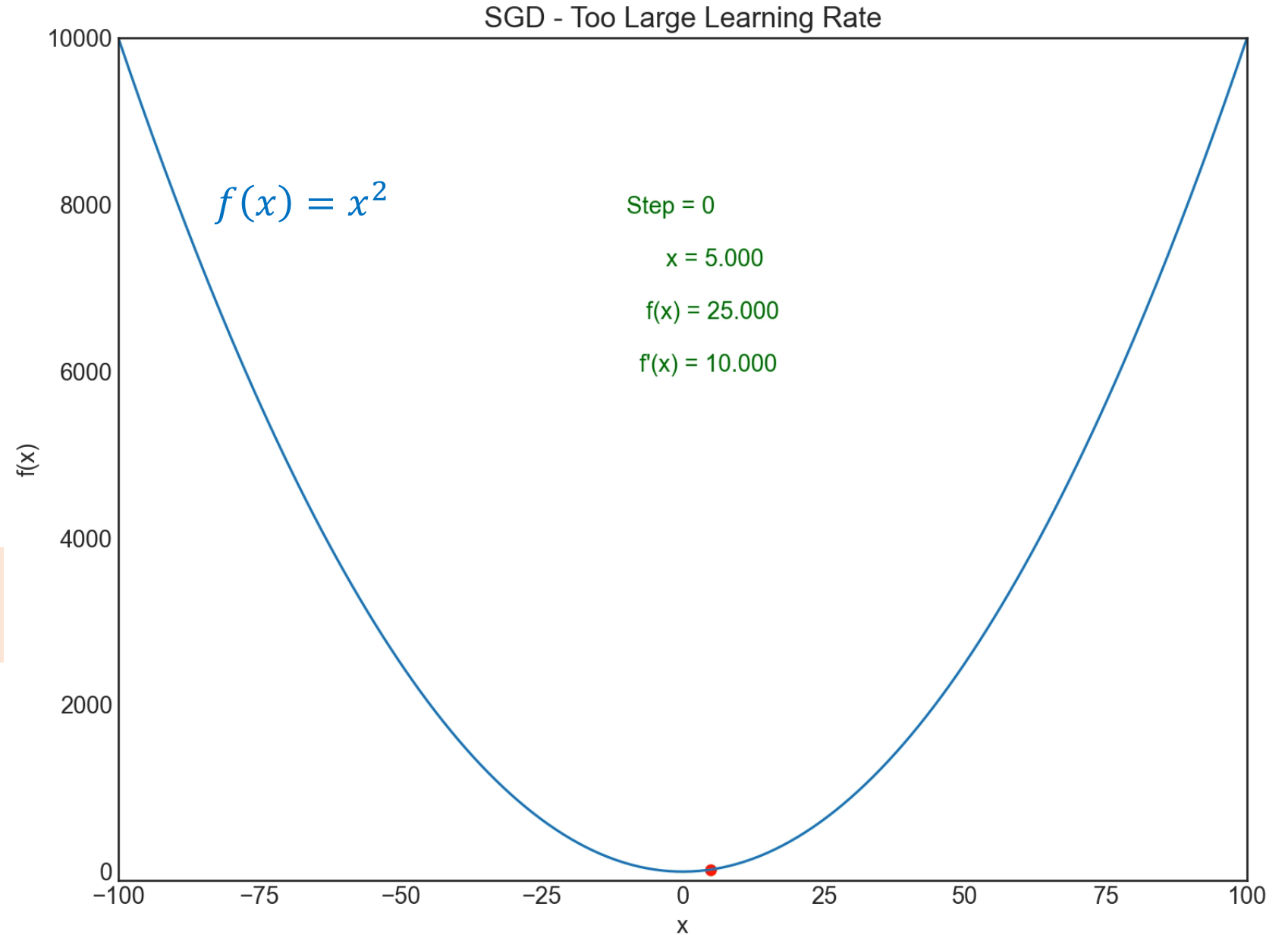
❖ Square function

Discussion

$$x_0 = 99.0$$

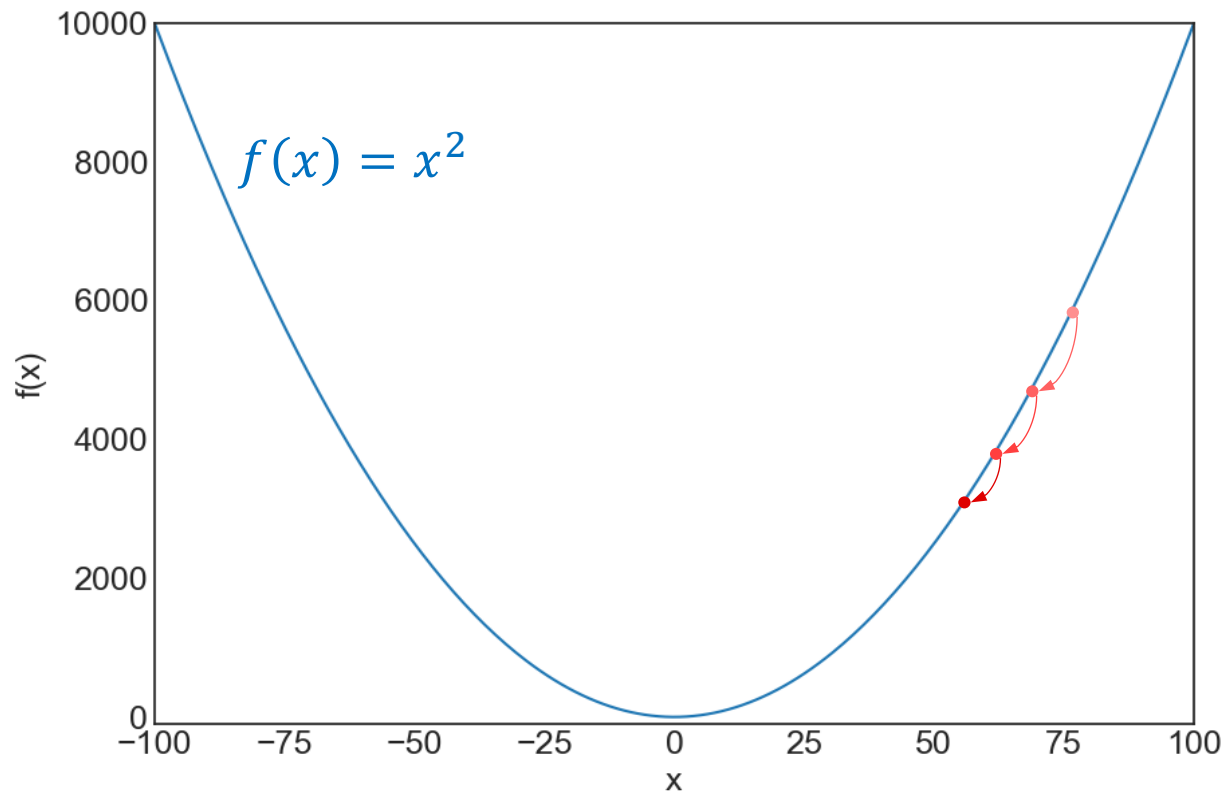
$$\eta = 1.1$$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

❖ Square function: Summary



- Given a function $f(x)$, find optimal x_{opt} so that $f(x_{\text{opt}})$ is minimum
- After an update, $f(x_{\text{new}}) \leq f(x_{\text{old}})$

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

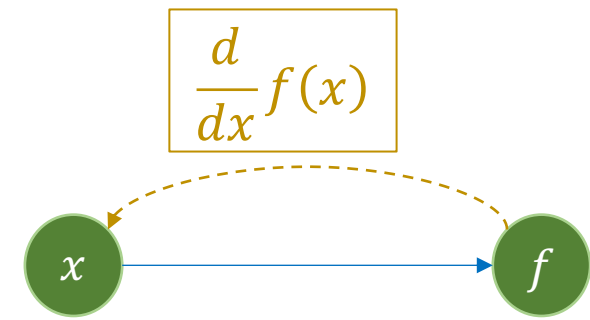
$$x_{30} = 0.069 \quad \eta = 0.1$$

$$f'(x_{30}) = 0.138$$

$$x_{31} = x_{30} - \eta f'(x_{30}) = 0.055$$

$$f'(x_{31}) = 0.11$$

$$x_{32} = x_{31} - \eta f'(x_{31}) = 0.044$$



What does $f(x)$ mean?

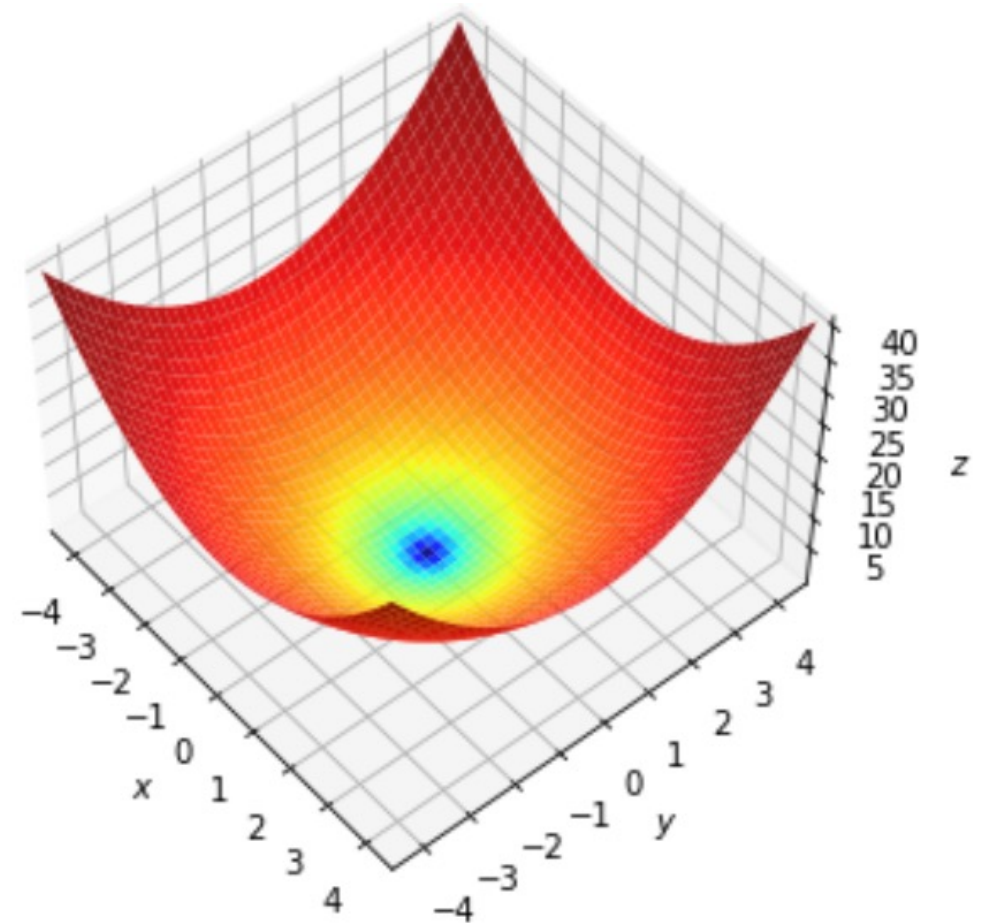
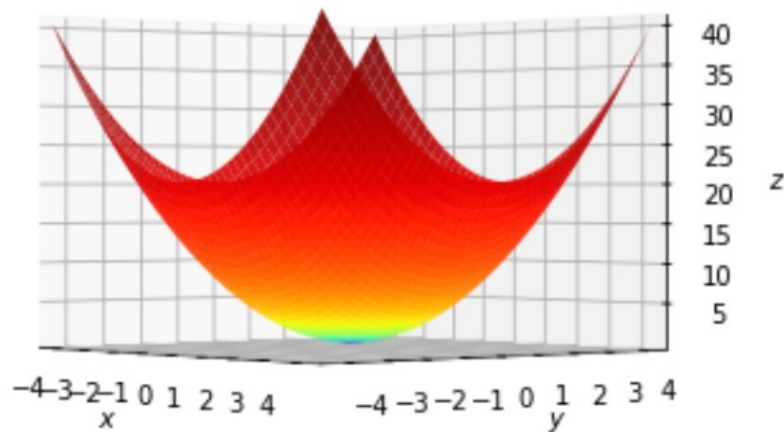
Observation 2

❖ Optimization: 2D function

$$f(x, y) = x^2 + y^2$$

$$-100 \leq x, y \leq 100$$

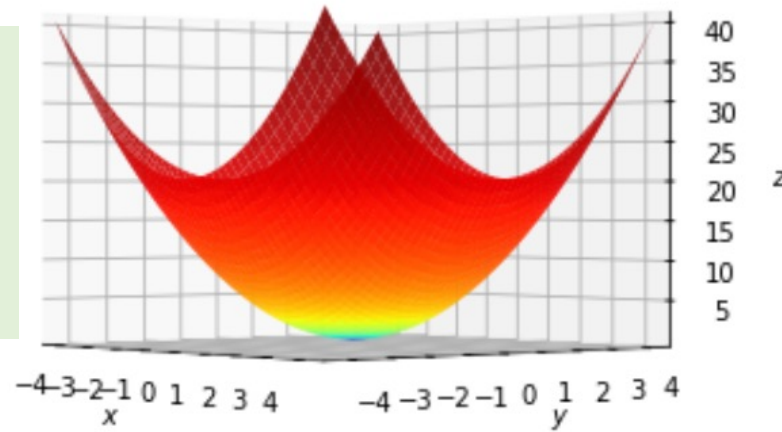
$$x, y \in \mathbb{N}$$



Optimization

❖ Optimization: 2D function

$$f(x, y) = x^2 + y^2$$
$$-100 \leq x, y \leq 100$$
$$x, y \in \mathbb{N}$$



$$x = x - \eta \frac{\partial f(x, y)}{\partial x}$$

$$y = y - \eta \frac{\partial f(x, y)}{\partial y}$$

$$x_0 = 6.0 \quad y_0 = 9.0 \quad \eta = 0.1$$

$$\frac{\partial f(x_0, y_0)}{\partial x} = 12 \quad \frac{\partial f(x_0, y_0)}{\partial y} = 18$$
$$x_1 = 4.8 \quad y_1 = 7.2$$

$$\frac{\partial f(x_1, y_1)}{\partial x} = 9.6 \quad \frac{\partial f(x_1, y_1)}{\partial y} = 14.4$$
$$x_2 = 3.84 \quad y_2 = 5.75$$

$$\frac{\partial f(x_2, y_2)}{\partial x} = 7.68 \quad \frac{\partial f(x_2, y_2)}{\partial y} = 11.51$$
$$x_3 = 3.07 \quad y_3 = 4.608$$

$$\frac{\partial f(x_3, y_3)}{\partial x} = 6.14 \quad \frac{\partial f(x_3, y_3)}{\partial y} = 9.21$$
$$x_4 = 2.45 \quad y_4 = 3.68$$

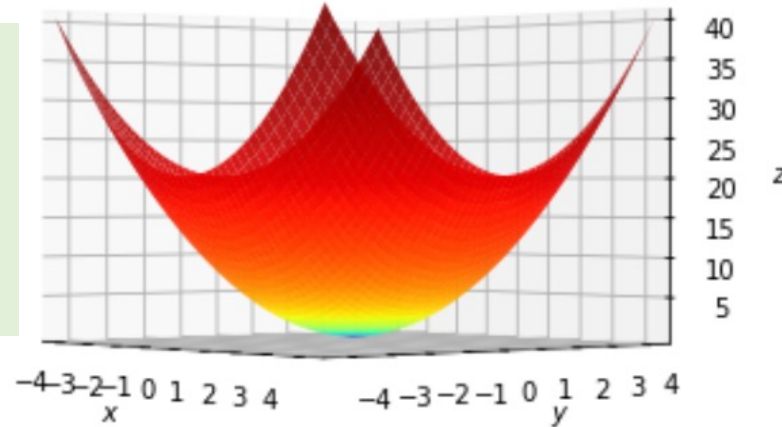
Optimization

❖ Optimization: 2D function

$$f(x, y) = x^2 + y^2$$

$$-100 \leq x, y \leq 100$$

$$x, y \in \mathbb{N}$$



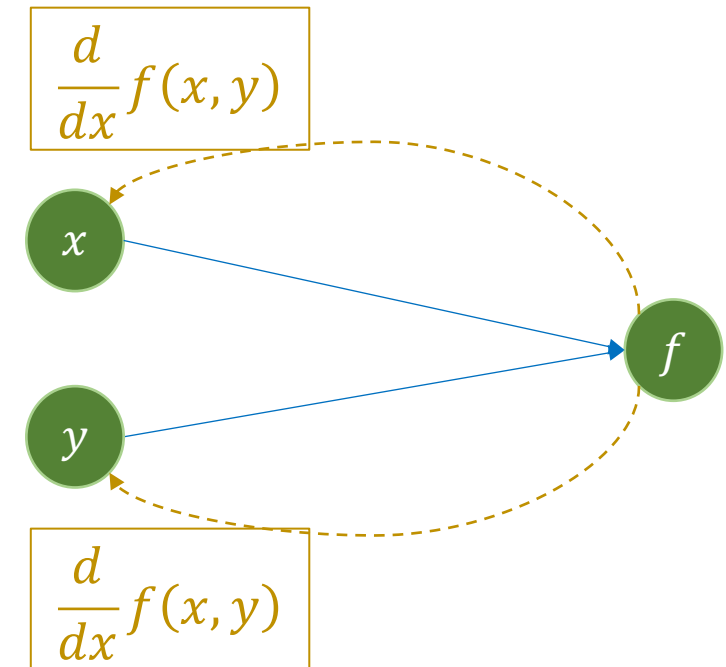
Summary:

- Given a function $f(x, y)$, find optimal $(x_{\text{opt}}, y_{\text{opt}})$ so that $f(x_{\text{opt}}, y_{\text{opt}})$ is minimum
- After an update, $f(x_{\text{new}}, y_{\text{new}}) \leq f(x_{\text{old}}, y_{\text{old}})$

What does $f(x, y)$ mean?

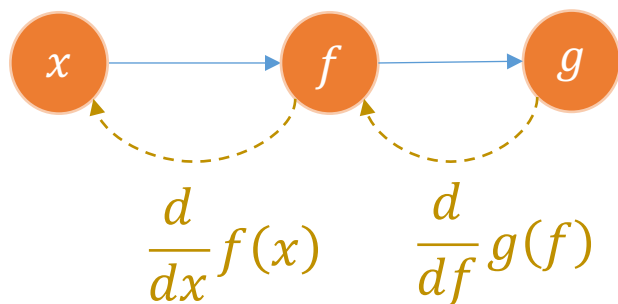
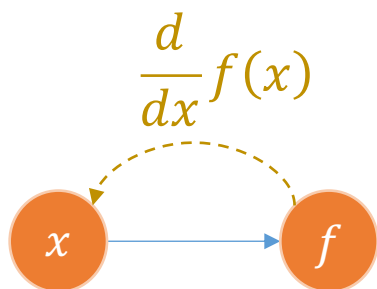
$$\begin{aligned} \frac{\partial f(x_2, y_2)}{\partial x} &= 7.68 & \frac{\partial f(x_2, y_2)}{\partial y} &= 11.51 \\ x_3 &= 3.07 & y_3 &= 4.608 \end{aligned}$$

$$\begin{aligned} \frac{\partial f(x_3, y_3)}{\partial x} &= 6.14 & \frac{\partial f(x_3, y_3)}{\partial y} &= 9.21 \\ x_4 &= 2.45 & y_4 &= 3.68 \end{aligned}$$

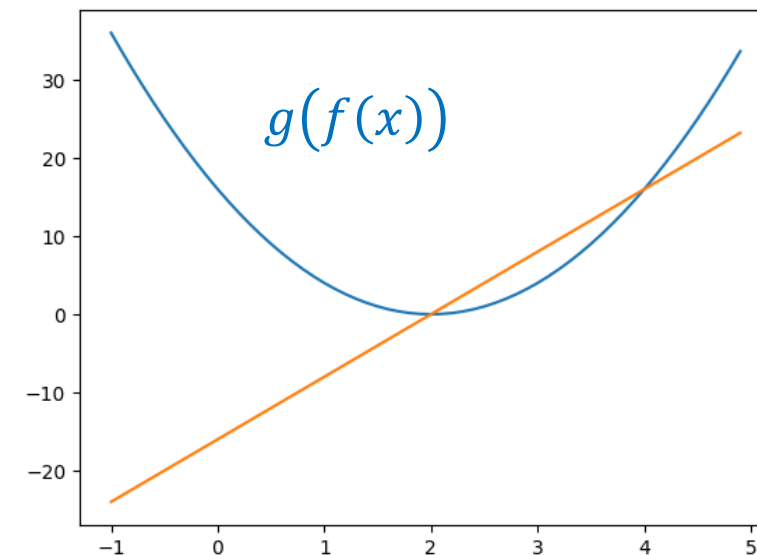
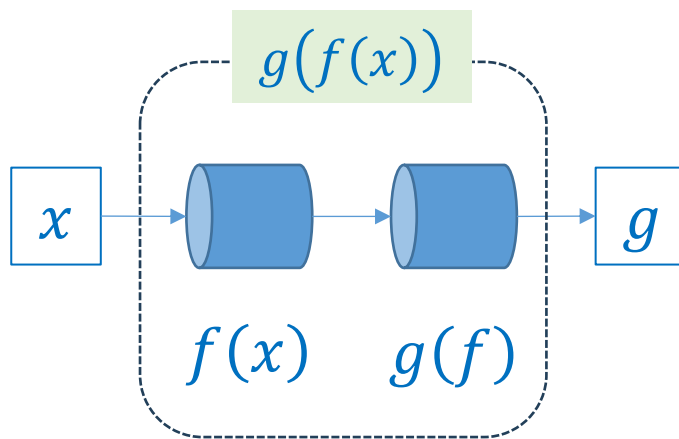
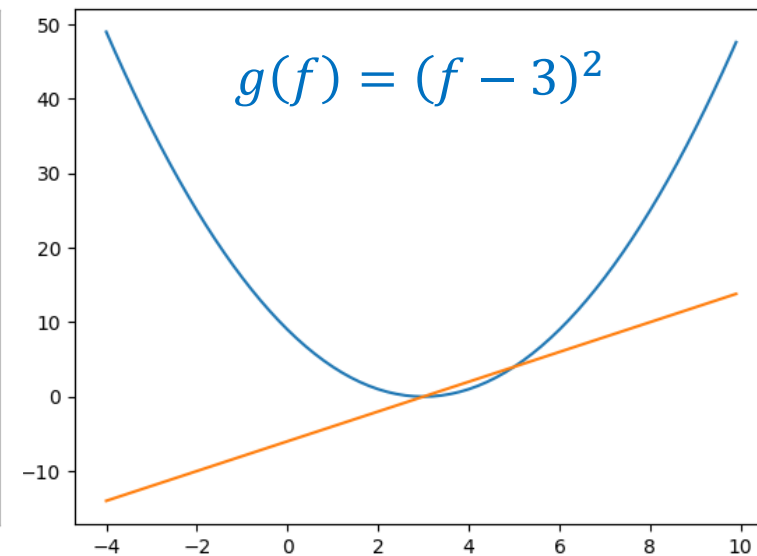
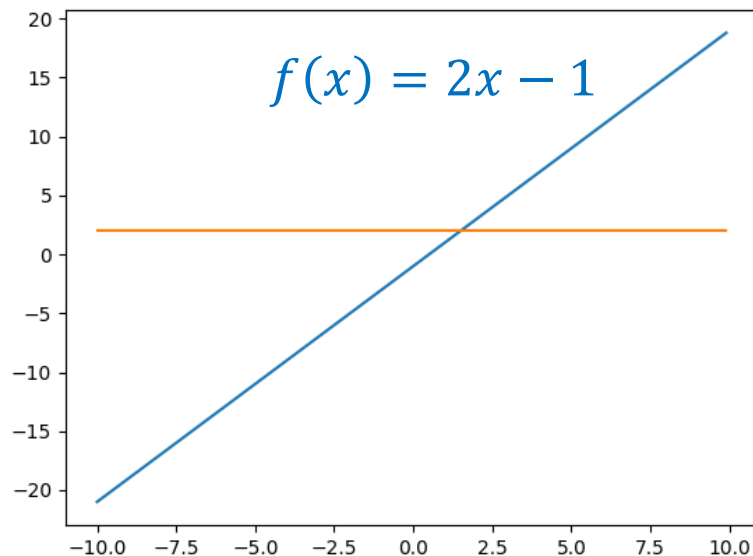


Observation 3

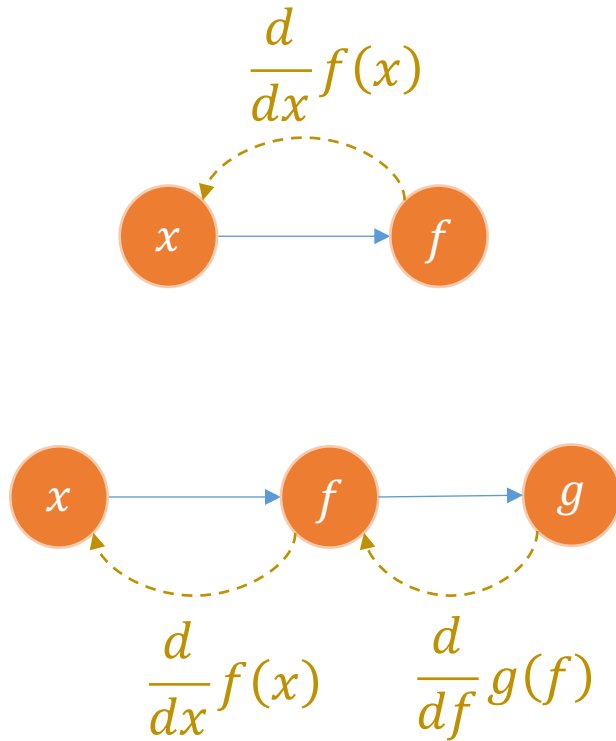
❖ For composite function



$$\frac{d}{dx} g(f(x)) = \left[\frac{d}{df} g(f) \right] * \left[\frac{d}{dx} f(x) \right]$$



❖ For composite function and chain rule



$$\frac{d}{dx} g(f(x)) = \left[\frac{d}{df} g(f) \right] * \left[\frac{d}{dx} f(x) \right]$$



$$f(x) = 2x - 1$$

$$g(f) = (f - 3)^2$$

$$f'(x) = 2$$

$$g'(f) = 2(f - 3)$$



$$\frac{dg}{dx} = \frac{dg}{df} \frac{df}{dx}$$

$$= 2(f - 3)2$$

$$= 4(2x - 1 - 3)$$

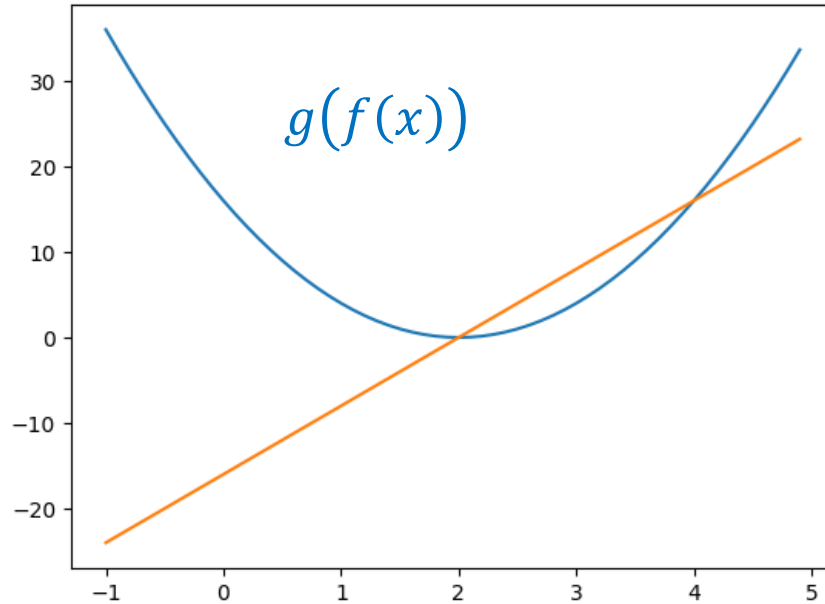
$$= 8x - 16$$

Implementation

$$f(x) = 2x - 1$$

$$g(f) = (f - 3)^2$$

$$\frac{dg}{dx} = 8x - 16$$



```
1 def fx(x):  
2     return 2*x - 1  
3  
4 def gf(f):  
5     return (f-3)**2  
6  
7 def dg_dx(x):  
8     return 8*x - 16
```

```
1 import random  
2  
3 # parameters  
4 lr = 0.1  
5  
6 # initialize x  
7 x = 60  
8  
9 old_loss = gf(fx(x)) # Logging  
10 print(f'old_loss: {old_loss}')
```

```
11  
12 # compute derivative  
13 dg_dx_value = dg_dx(x)  
14  
15 # update  
16 x = x - lr*dg_dx_value  
17  
18 new_loss = gf(fx(x)) # Logging  
19 print(f'new_loss: {new_loss}')
```

```
old_loss: 13456
```

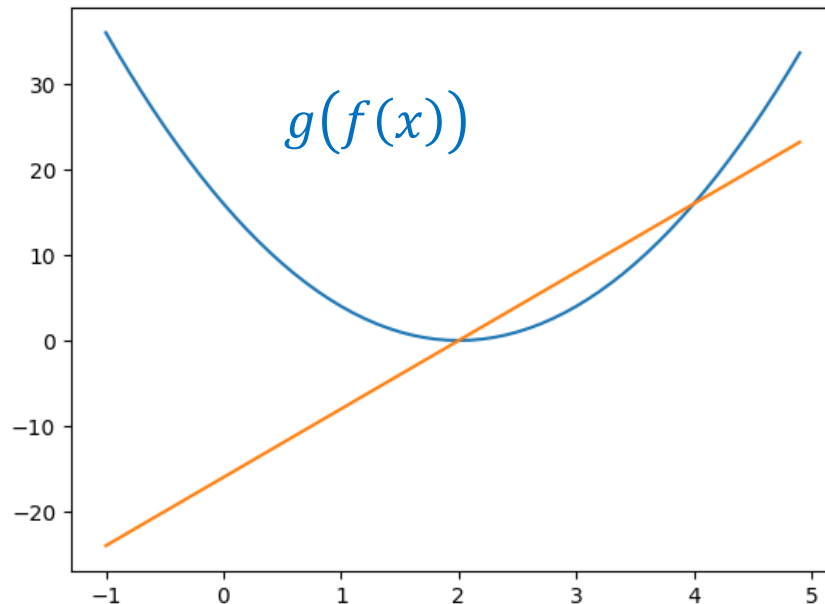
```
new_loss: 538.23999999999994
```

Implementation

$$f(x) = 2x - 1$$

$$g(f) = (f - 3)^2$$

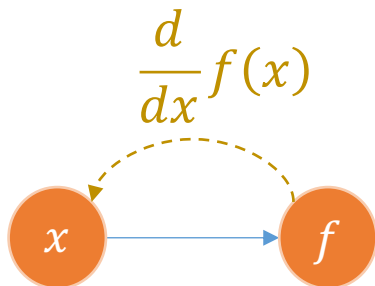
$$\frac{dg}{dx} = 8x - 16$$



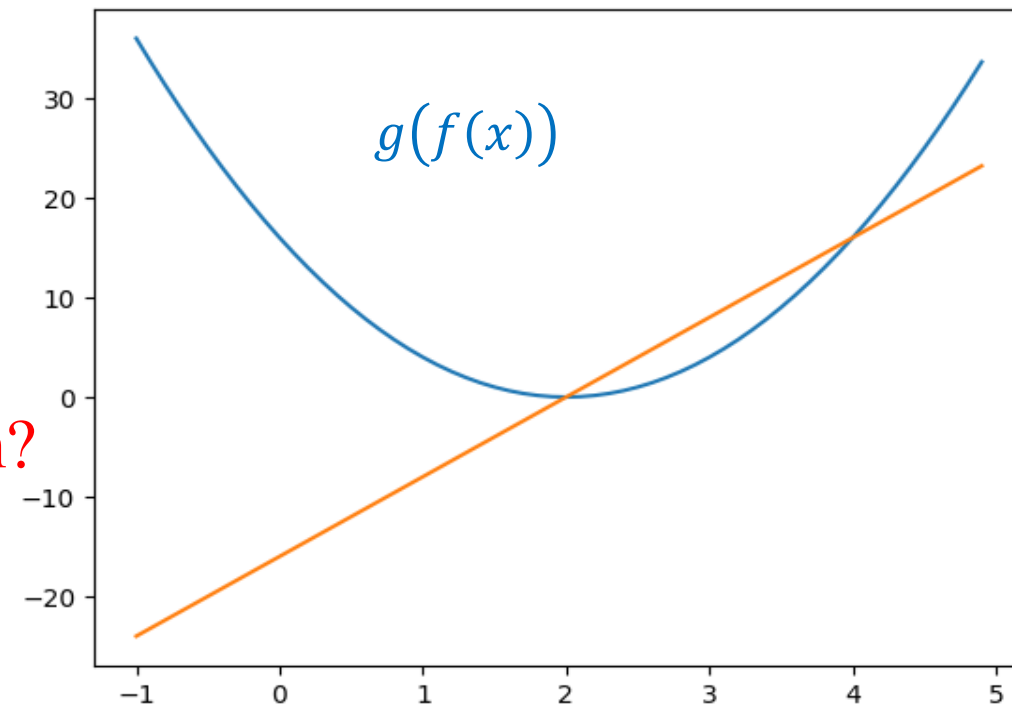
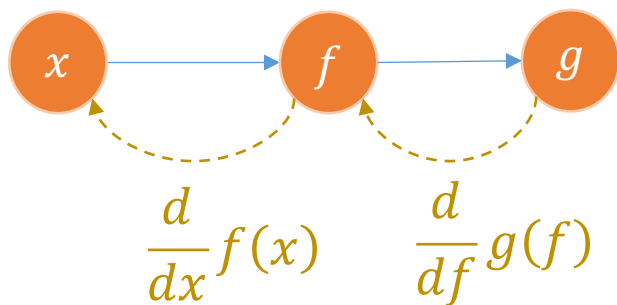
```
1 def fx(x):  
2     return 2*x - 1  
3  
4 def gf(f):  
5     return (f-3)**2  
6  
7 def dg_dx(x):  
8     return 8*x - 16
```

```
1 import random  
2  
3 # parameters  
4 num_steps = 5  
5 lr = 0.1  
6  
7 # set x randomly  
8 x = random.randint(-100, 100)  
9  
10 for _ in range(num_steps):  
11     # logging  
12     loss = gf(fx(x))  
13  
14     # compute derivative  
15     dg_dx_value = dg_dx(x)  
16  
17     # update  
18     x = x - lr*dg_dx_value
```

❖ For composite function: Summary

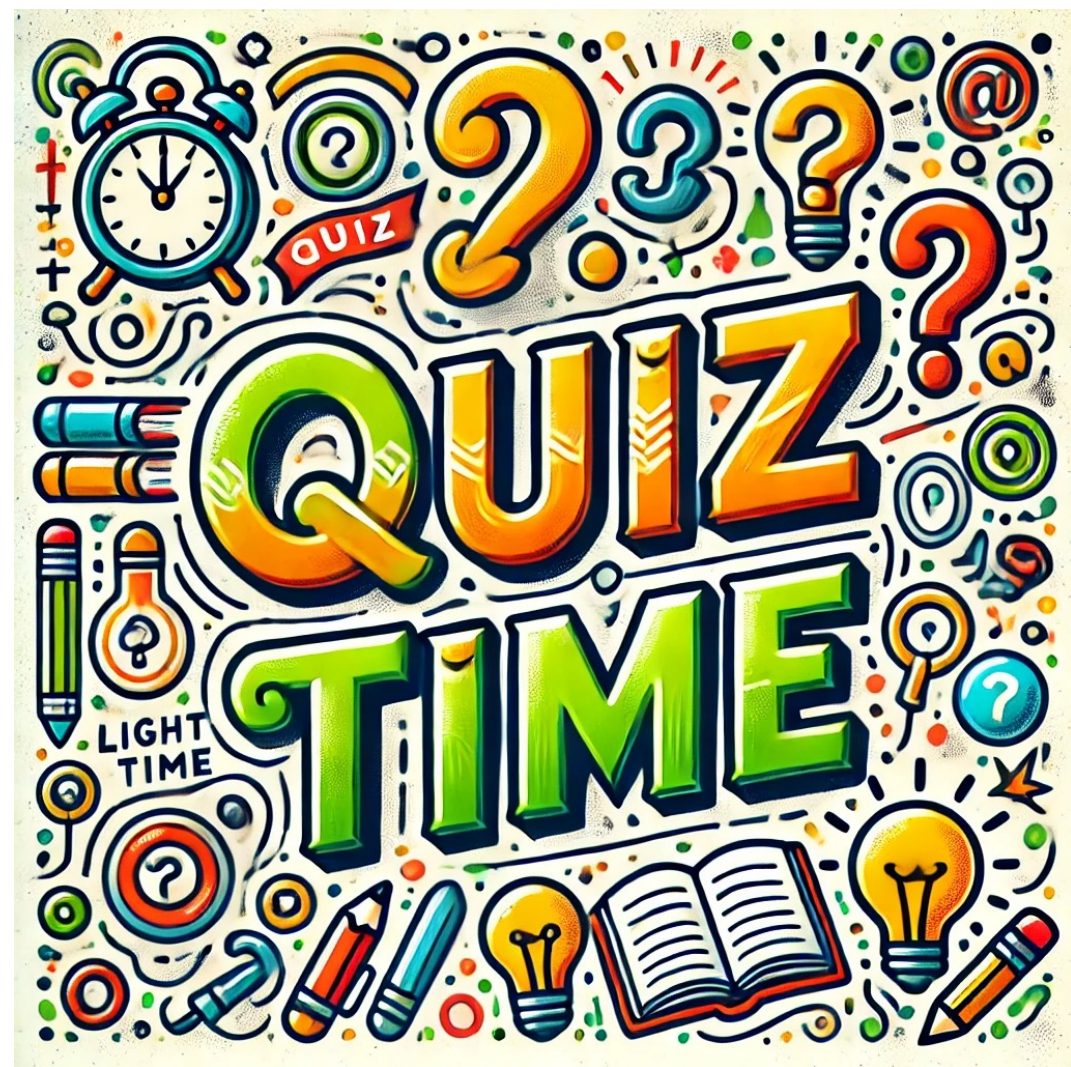


What does $g(f(x))$ mean?



- Given a function $g(f(x))$, find optimal x_{opt} so that $g(f(x_{\text{opt}}))$ is minimum
- After an update, $g(f(x_{\text{new}})) \leq g(f(x_{\text{old}}))$

$$\frac{d}{dx} g(f(x)) = \left[\frac{d}{df} g(f) \right] * \left[\frac{d}{dx} f(x) \right]$$



Outline

SECTION 1

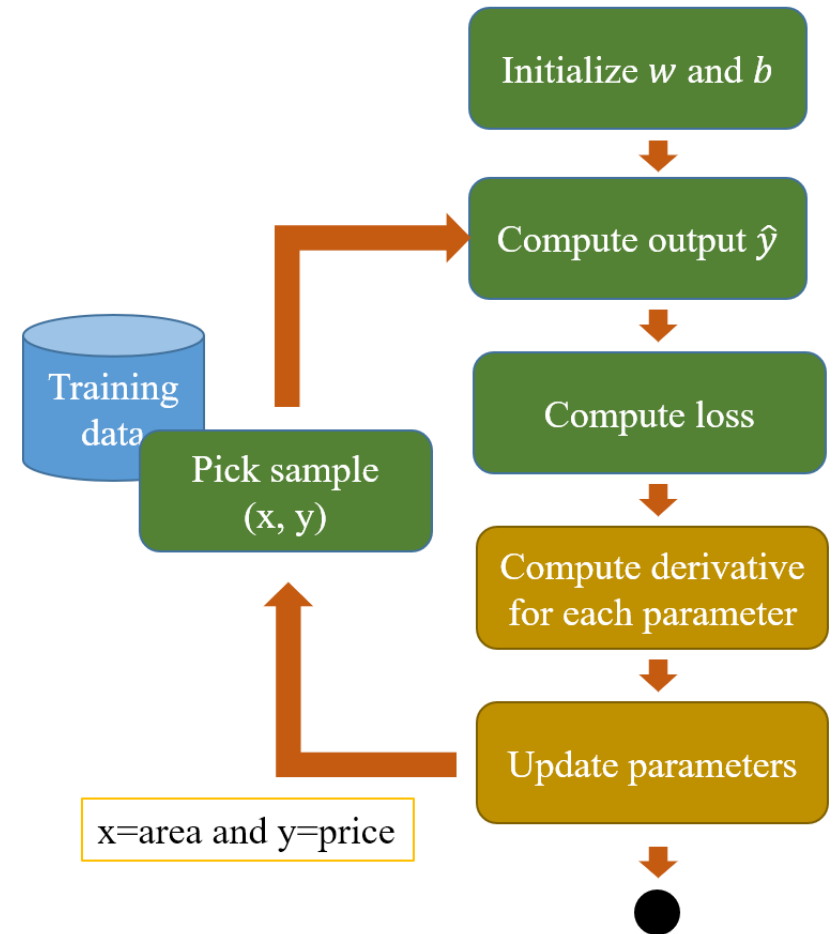
Gradient descent

SECTION 2

Linear Regression

SECTION 3

Vectorization



Linear Regression

Introduction

Feature		Label	
	area	price	
	6.7	9.1	
	4.6	5.9	
	3.5	4.6	
	5.5	6.7	

House price data

Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

if area=6.0, price=?

if TV=55.0, Radio=34.0,
and Newspaper=62.0,
price=?

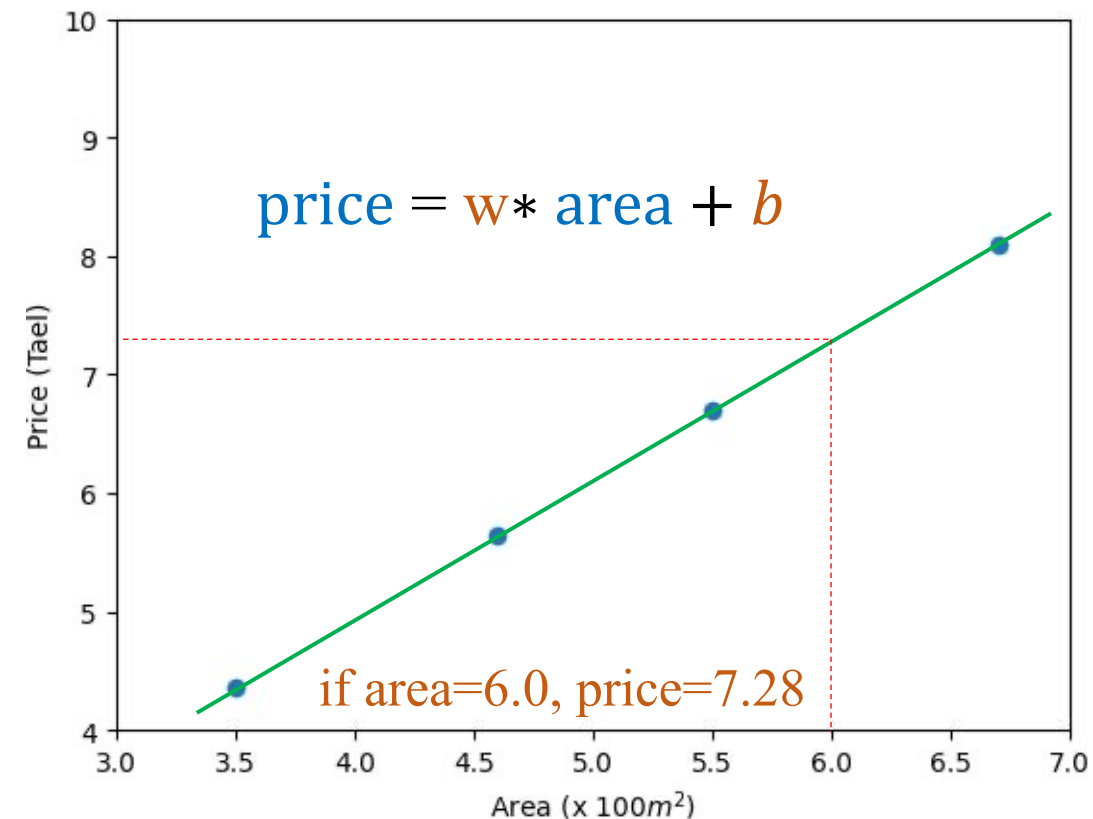
Features													Label
crim	↕ zn	↕ indus	↕ chas	↕ nox	↕ rm	↕ age	↕ dis	↕ rad	↕ tax	↕ ptratio	↕ black	↕ lstat	↕ medv
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9

Boston House Price Data

House Price Prediction

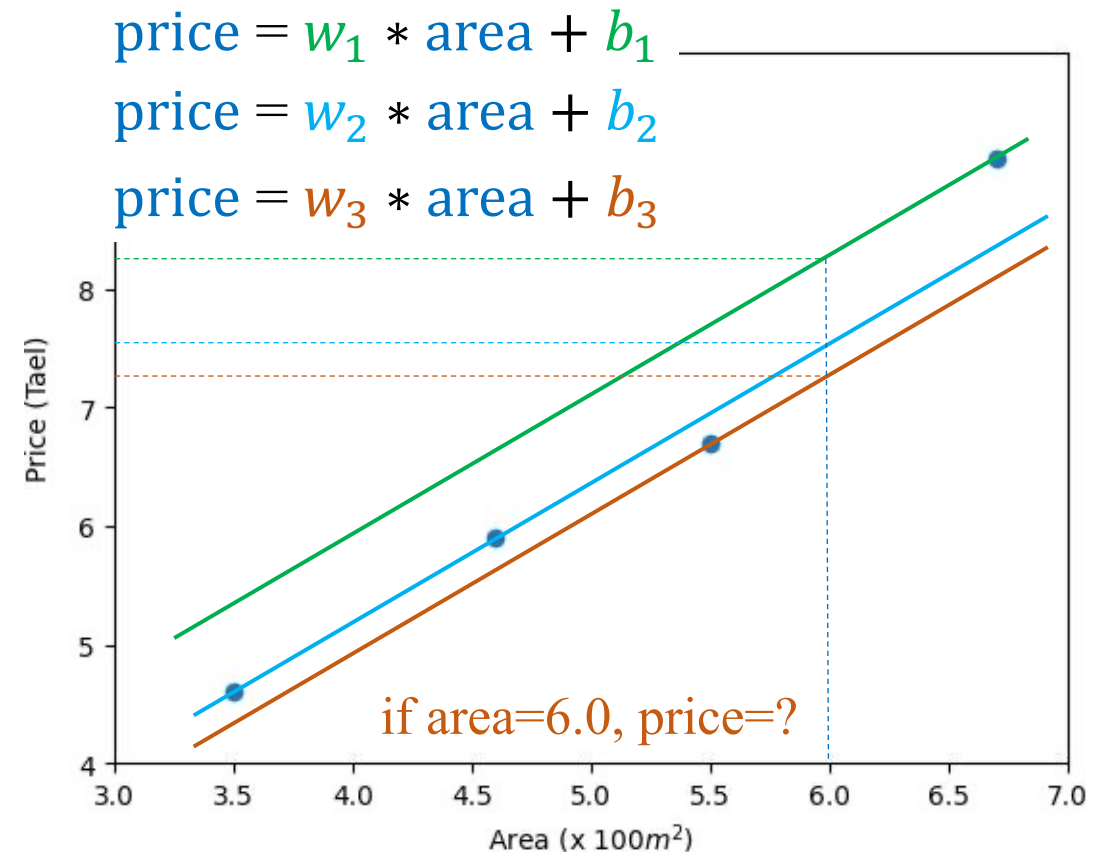
House price data

Feature	Label
area	price
6.7	8.1
4.6	5.6
3.5	4.3
5.5	6.7



Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

House price data



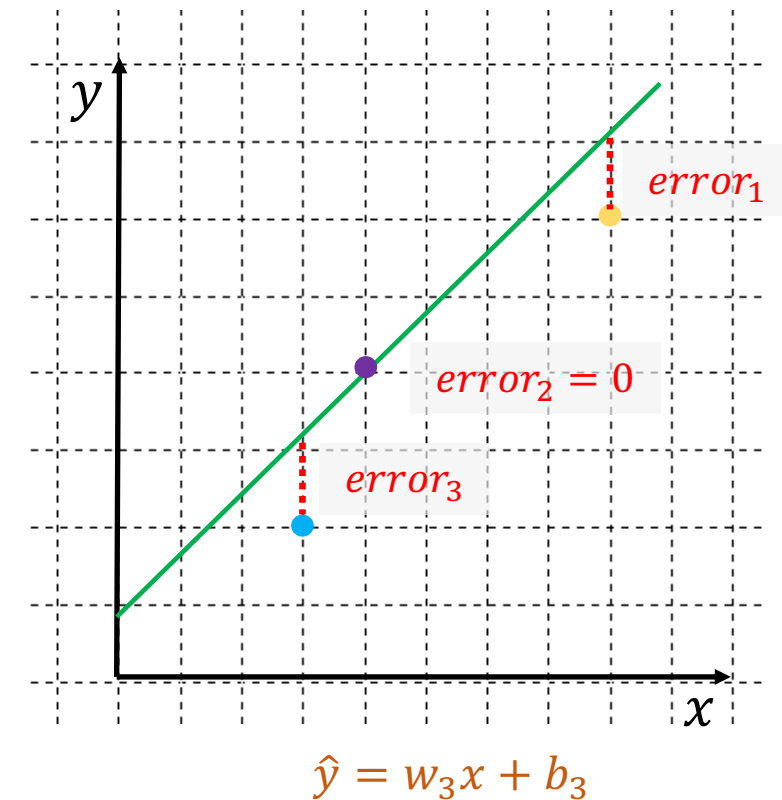
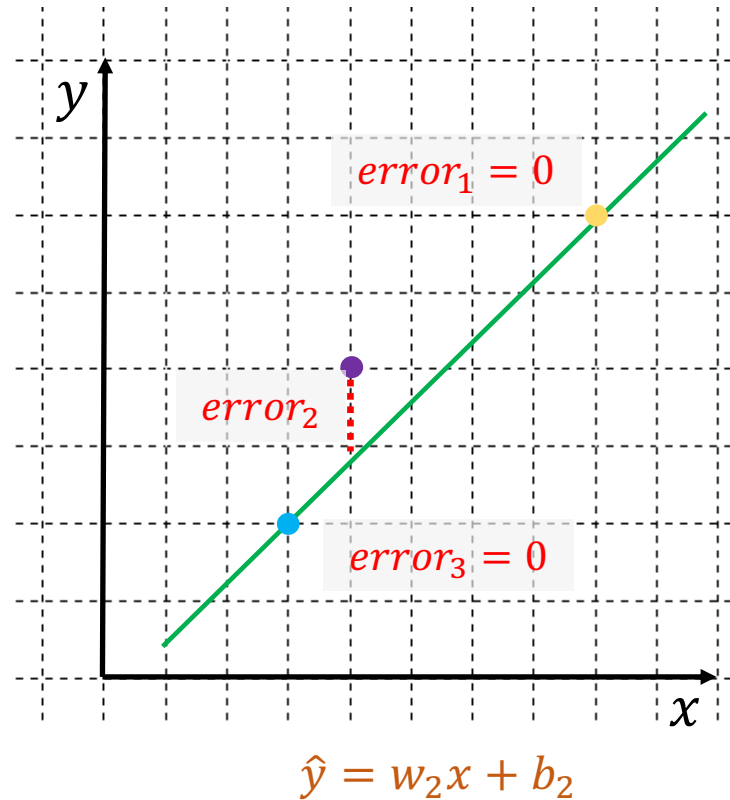
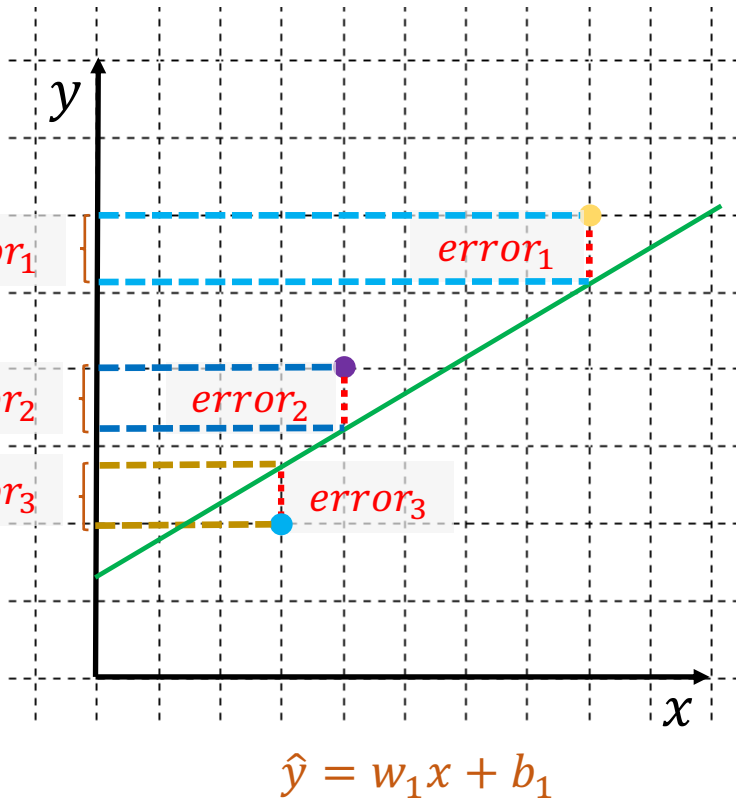
Linear Regression

32

❖ Area-based house price prediction



$$error_i = distance(\hat{y}_i, y_i)$$



Find w and b whose model has the smallest error, where $error = \frac{1}{N} \sum_i error_i$ **How?**

❖ Area-based house price prediction

$$\text{predicted_price} = \overset{\text{weight}}{w} * \text{area} + \overset{\text{bias}}{b}$$

$$\text{error} = (\text{predicted_price} - \text{real_price})^2$$

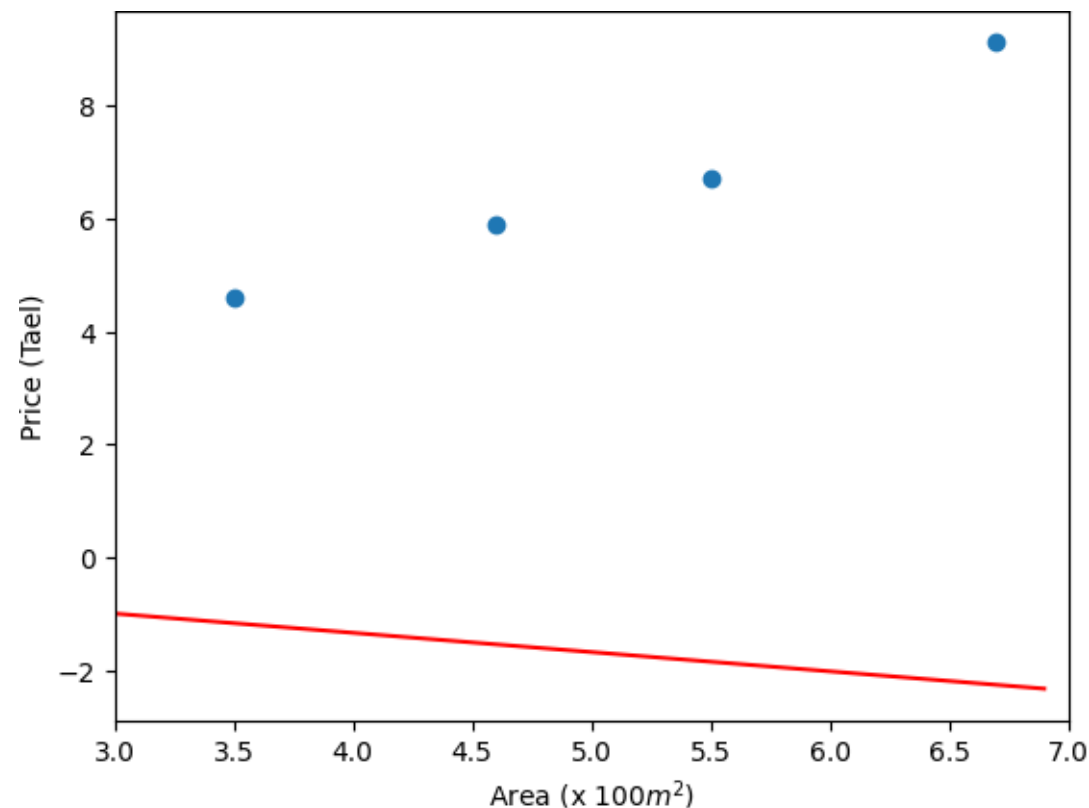
$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

area	price	predicted	error
6.7	9.1	-2.238	128.55
4.6	5.9	-1.524	55.11
3.5	4.6	-1.15	33.06
5.5	6.7	-1.83	72.76

$$w = -0.34$$

$$b = 0.04$$



❖ Area-based house price prediction

$$\text{predicted_price} = w * \text{area} + b$$

$$\text{error} = (\text{predicted_price} - \text{real_price})^2$$

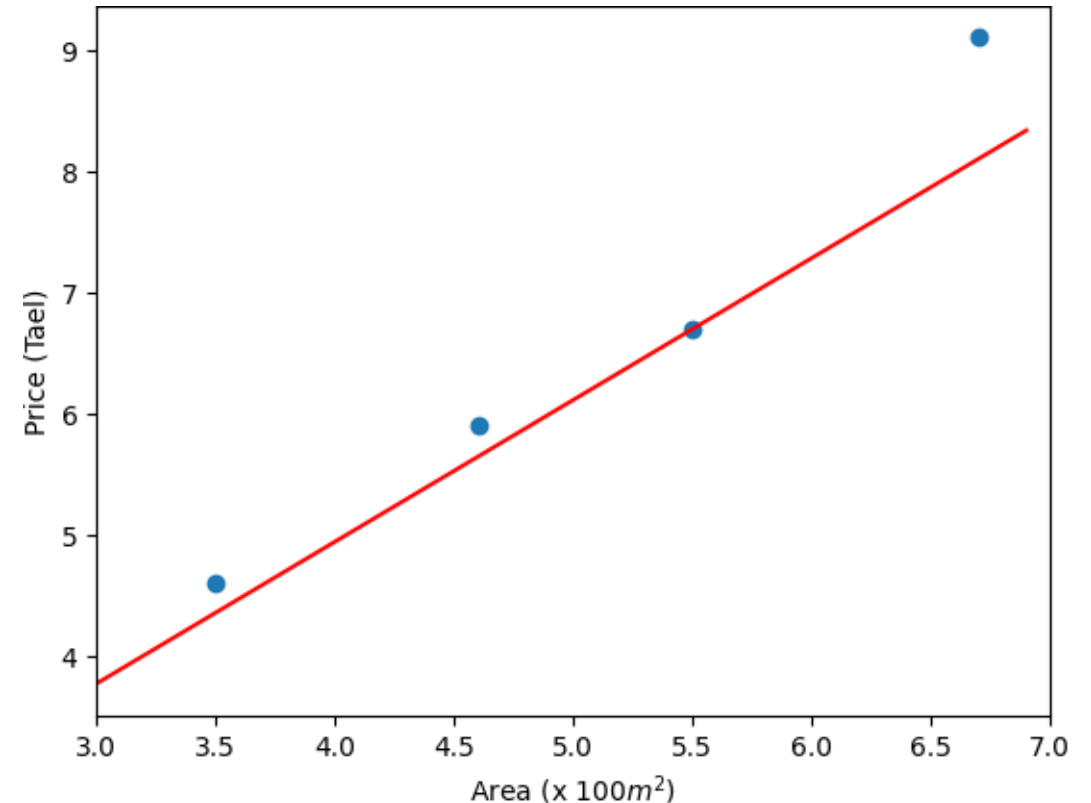
$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

area	price	predicted	error
6.7	9.1	8.099	1.002
4.6	5.9	5.642	0.066
3.5	4.6	4.355	0.06
5.5	6.7	6.695	0.00002

$$w = 1.17$$

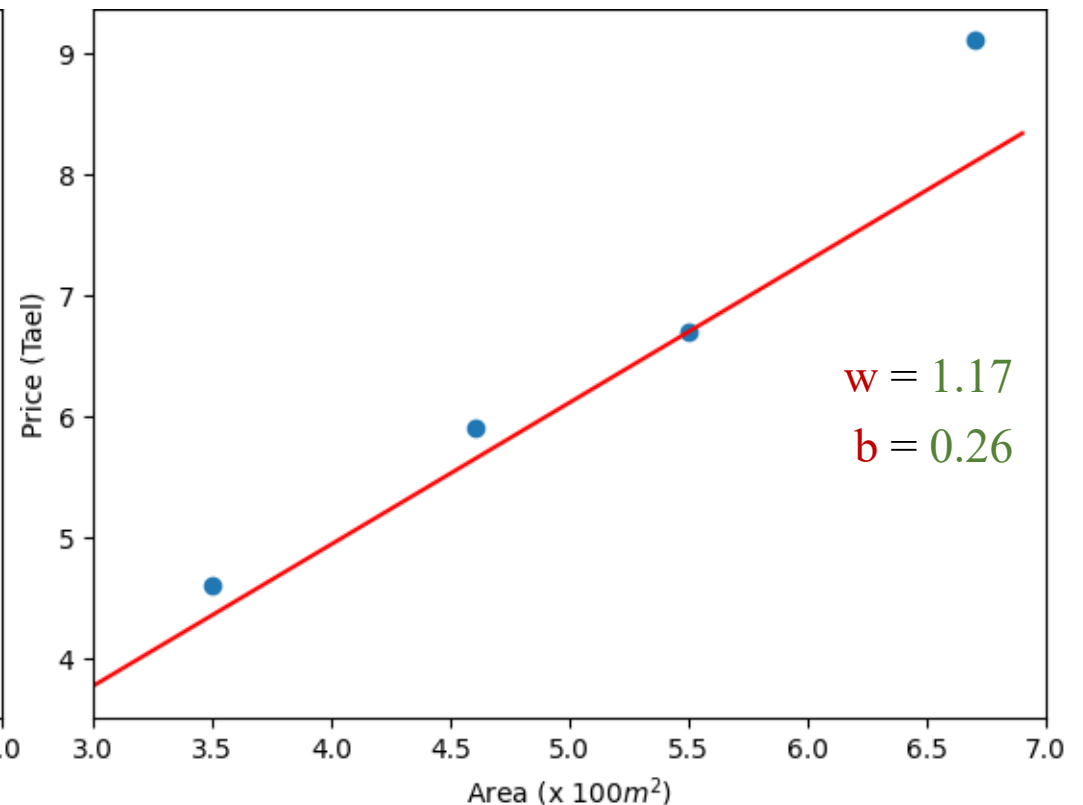
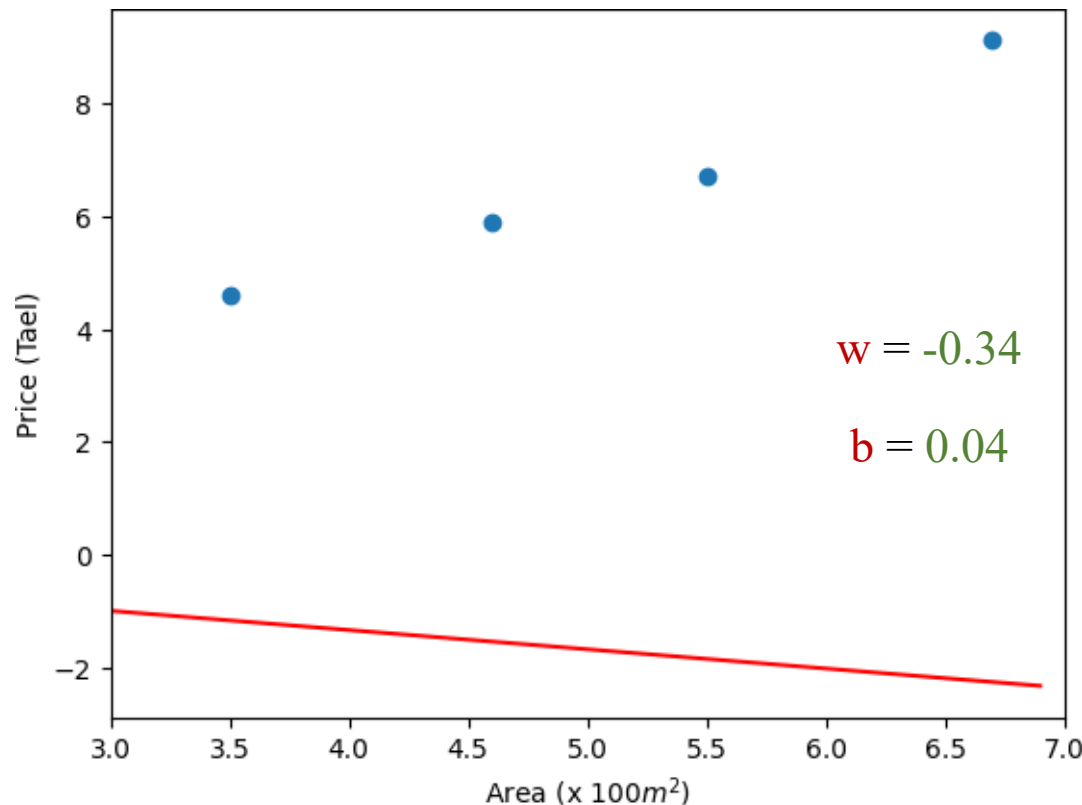
$$b = 0.26$$



❖ Area-based house price prediction

$$\hat{y}_i = wx_i + b$$
$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

How to change w and b
so that $L(\hat{y}_i, y_i)$ reduces



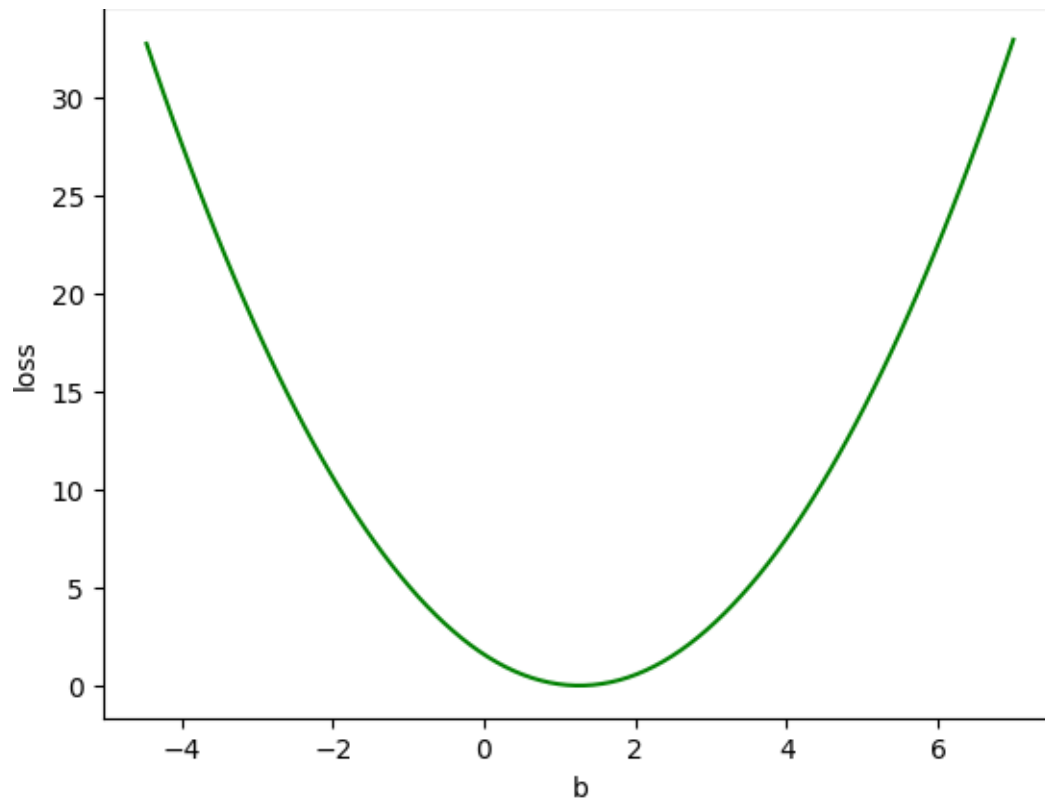
Linear Regression

❖ Understanding the loss function

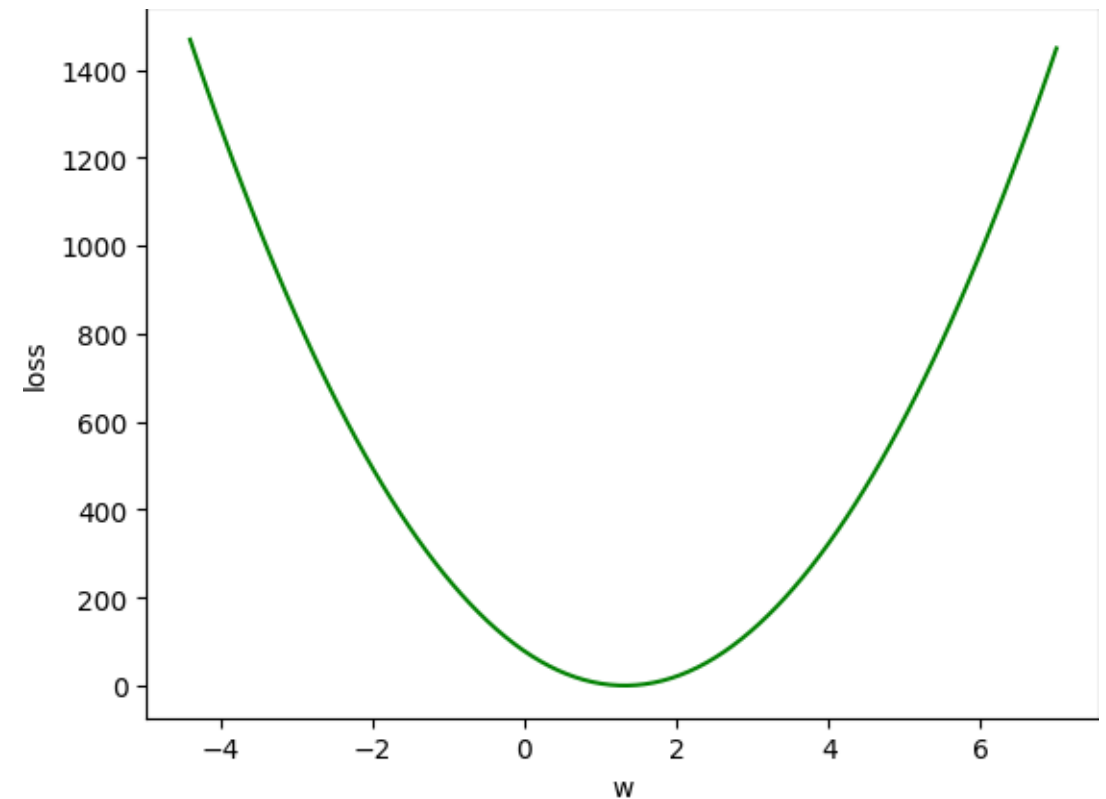
$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

How to change w and b so that $L(\hat{y}_i, y_i)$ reduces



Different b values with a fixed w value



Different w values with a fixed b value

Linear equation

$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,

w and b are parameters

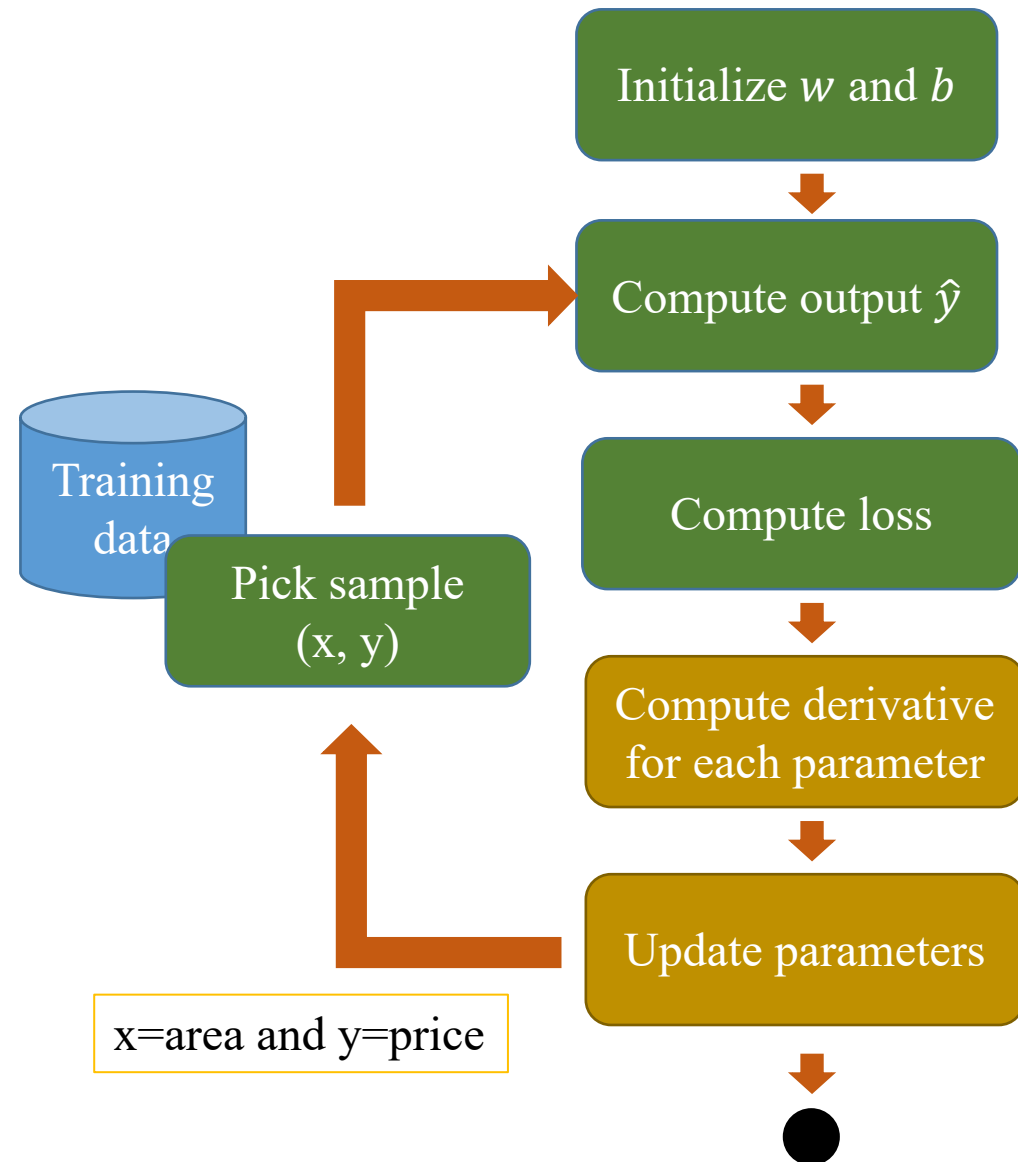
and x is an input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y

Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$



Linear equation

$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,

w and b are parameters

and x is an input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y

Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

Find better w and b

Use gradient descent to minimize the loss function

Compute derivate for each parameter

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y)$$

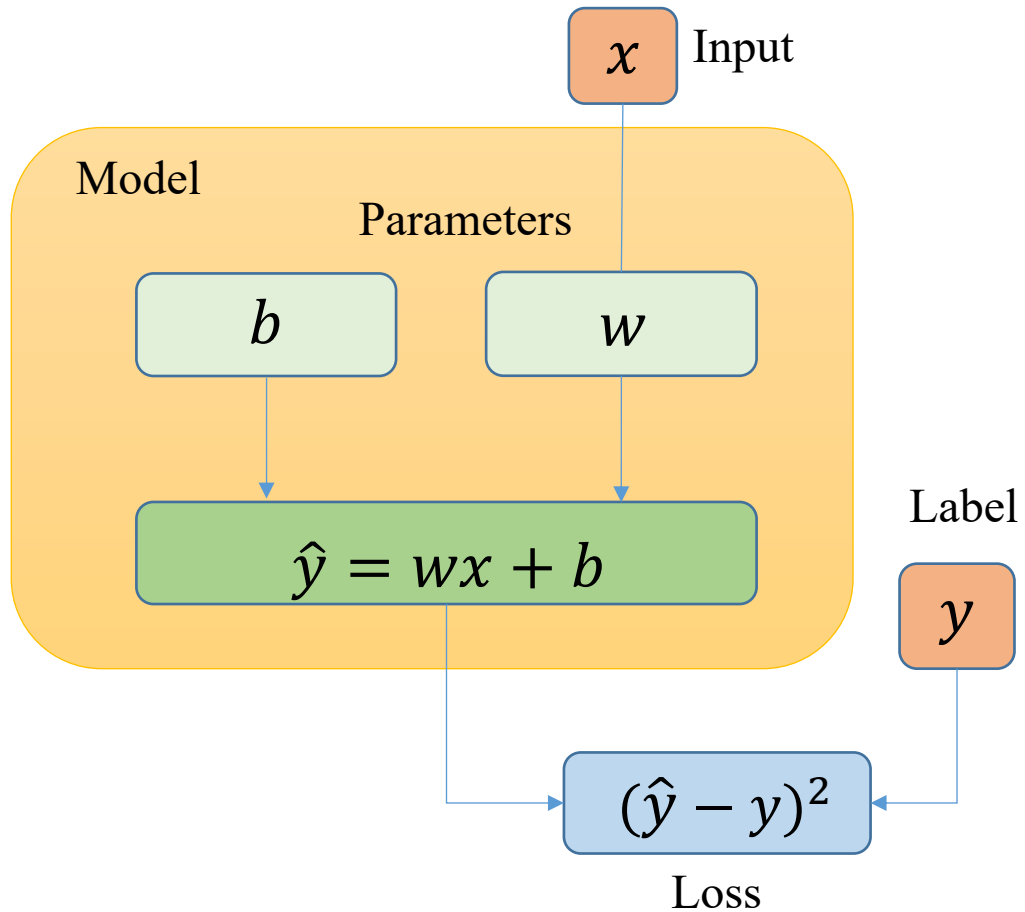
Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

❖ Simple example

Diagram



Cheat sheet

Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

Compute the loss

$$L = (\hat{y} - y)^2$$

Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

Update parameters

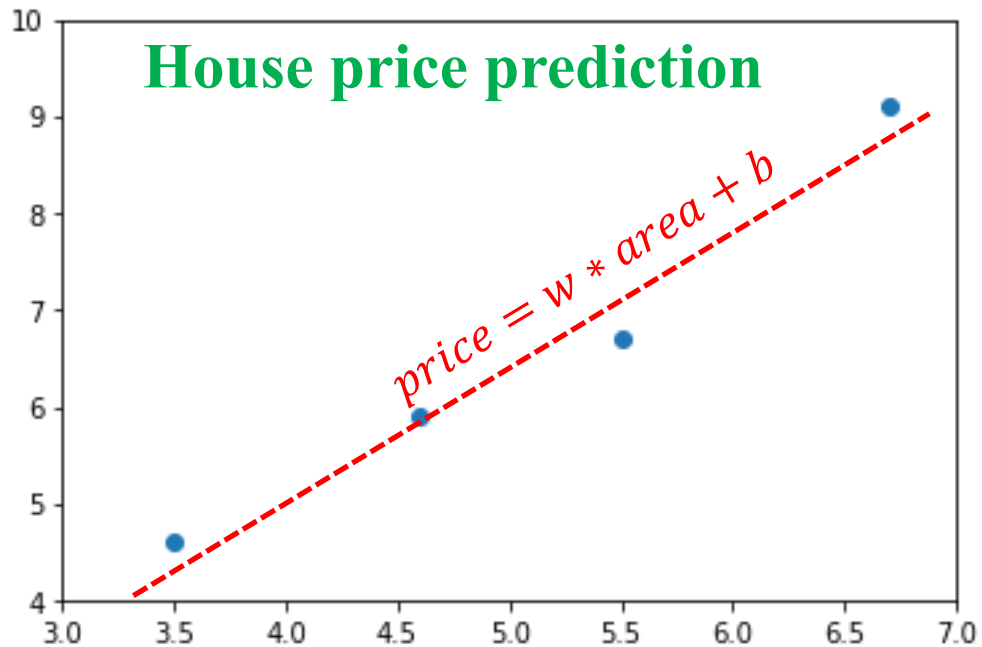
$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

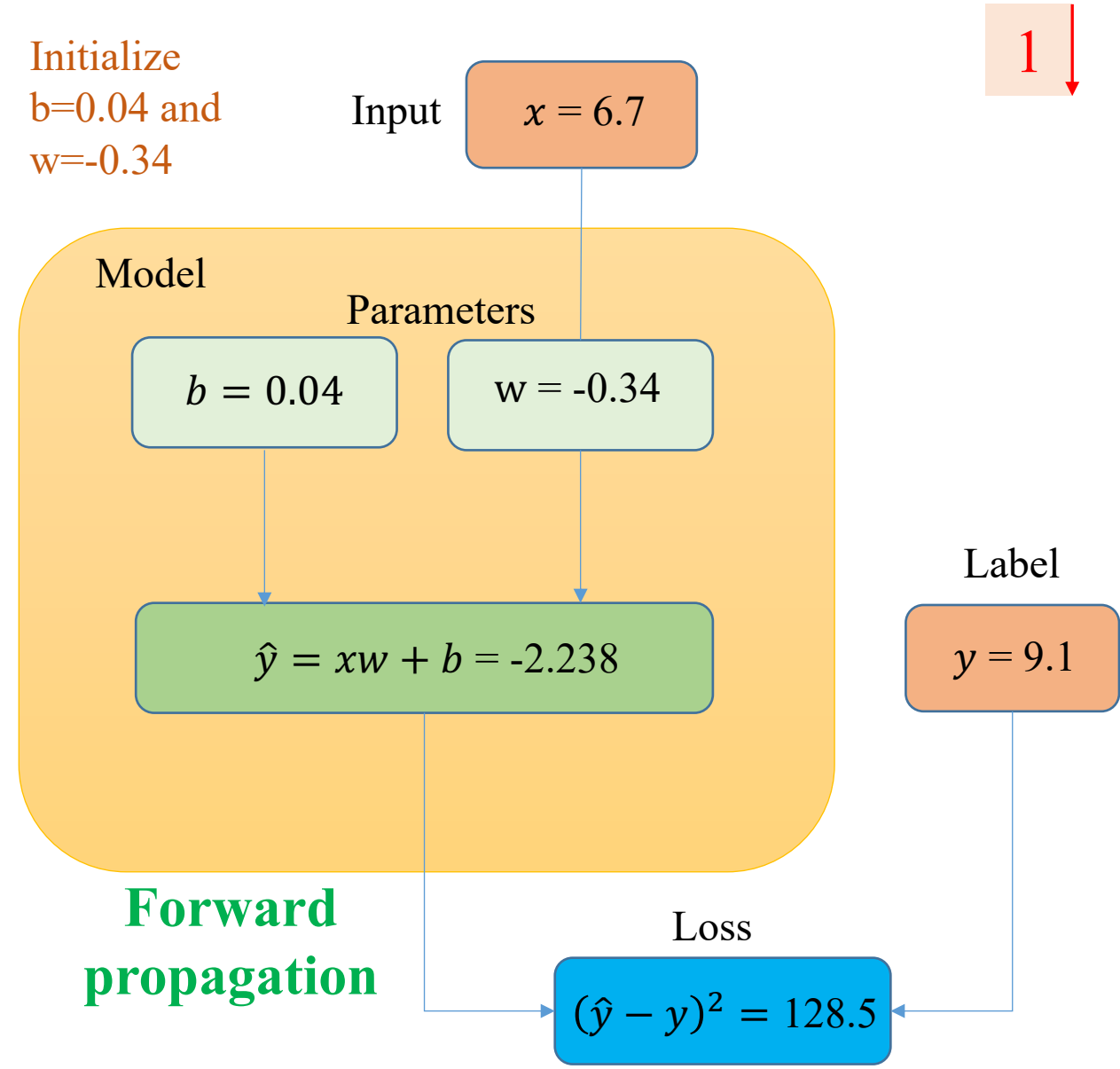
Linear Regression

Given
sample
data

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7



Initialize
 $b=0.04$ and
 $w=-0.34$



Linear Regression

41

2

Input

$x = 6.7$

Backpropagation

$\eta = 0.01$

Model

Parameters

$b = 0.26676$

$w = 1.17929$

$$b = b - \eta \frac{\partial L}{\partial b}$$

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$\hat{y} = xw + b = -2.238$$

$$\begin{aligned} \frac{\partial L}{\partial w} &= 2x(\hat{y} - y) \\ &= -151.9292 \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial b} &= 2(\hat{y} - y) \\ &= -22.676 \end{aligned}$$

Label

$y = 9.1$

Loss

$$(\hat{y} - y)^2 = 128.5$$

3

Input

$x = 6.7$

Forward propagation

Model

Parameters

$b = 0.26676$

$w = 1.17929$

$$\hat{y} = xw + b = 8.168$$

Label

$y = 9.1$

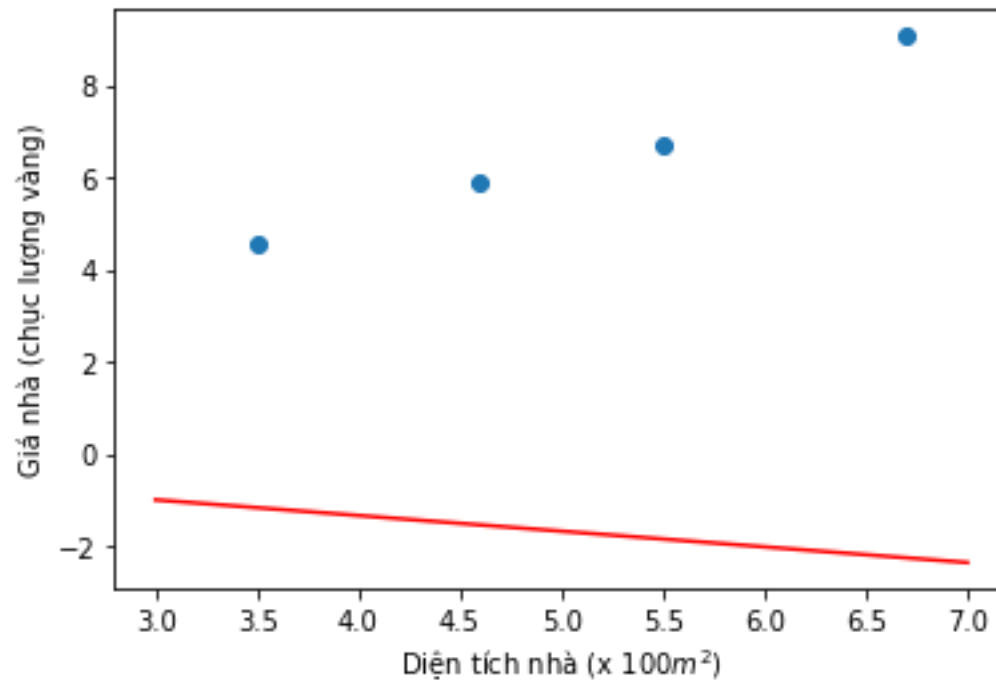
Loss

$$(\hat{y} - y)^2 = 0.868$$

New w and b help
the loss reduce

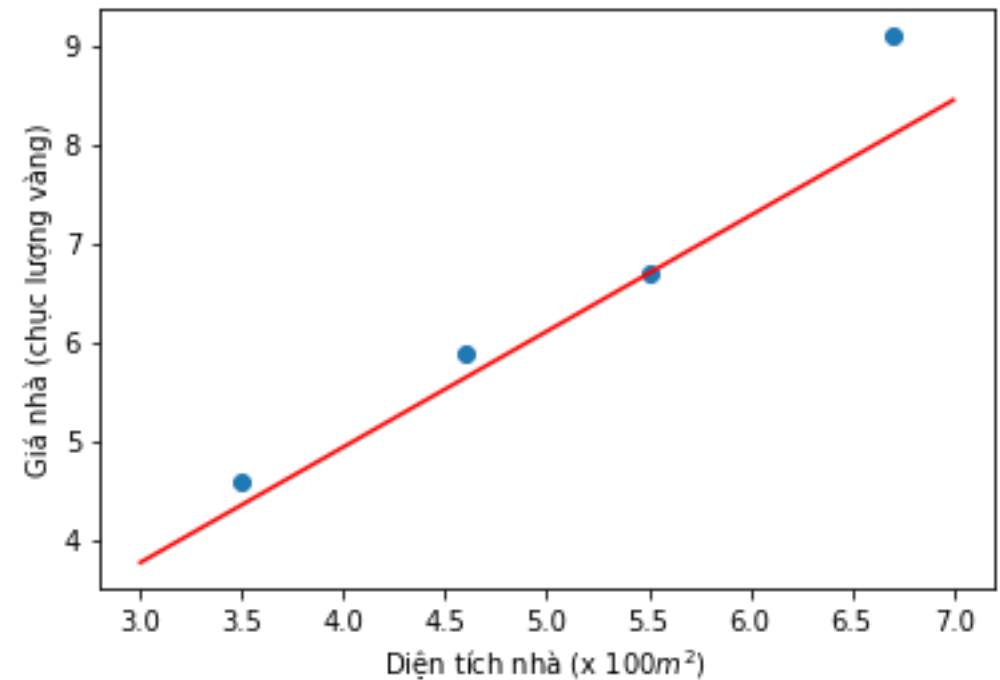
❖ Simple example

Model prediction before and after the first update



$w = -0.34$ $b = 0.04$ $L = 128.55$

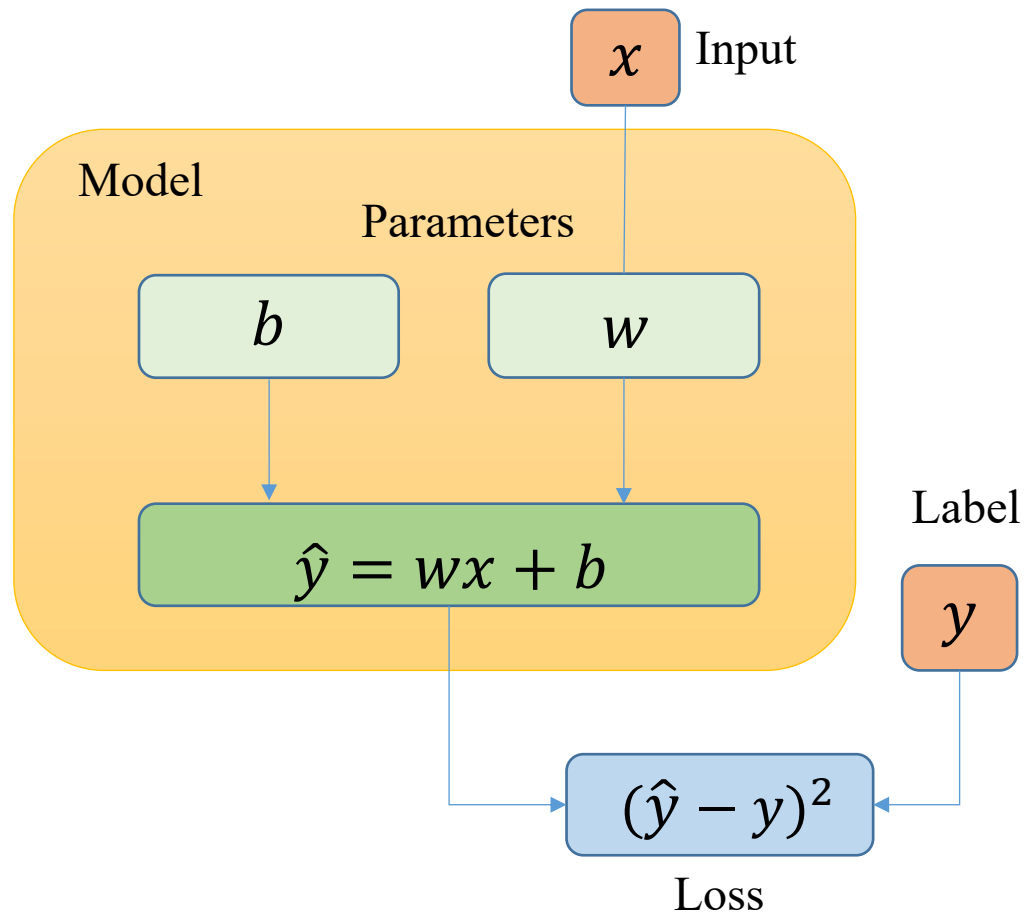
Before updating



$w = 1.179292$ $b = 0.26676$ $L = 0.868$

After updating

❖ Summary (simple version)



1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

❖ One-sample training

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

column index=0 column index=1

```
# data preparation
import numpy as np
import matplotlib.pyplot as plt

data = np.genfromtxt('data.csv',
                    | | | | | delimiter=',')

x_data = data[:,0]
y_data = data[:,1]
N = data.shape[0]

print(f'areas: {x_data}')
print(f'prices: {y_data}')
print(f'data_size: {N}')
```

✓ 0.2s

```
areas: [6.7 4.6 3.5 5.5]
prices: [9.1 5.9 4.6 6.7]
data_size: 4
```

❖ House price prediction

❖ One-sample training

Demo

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

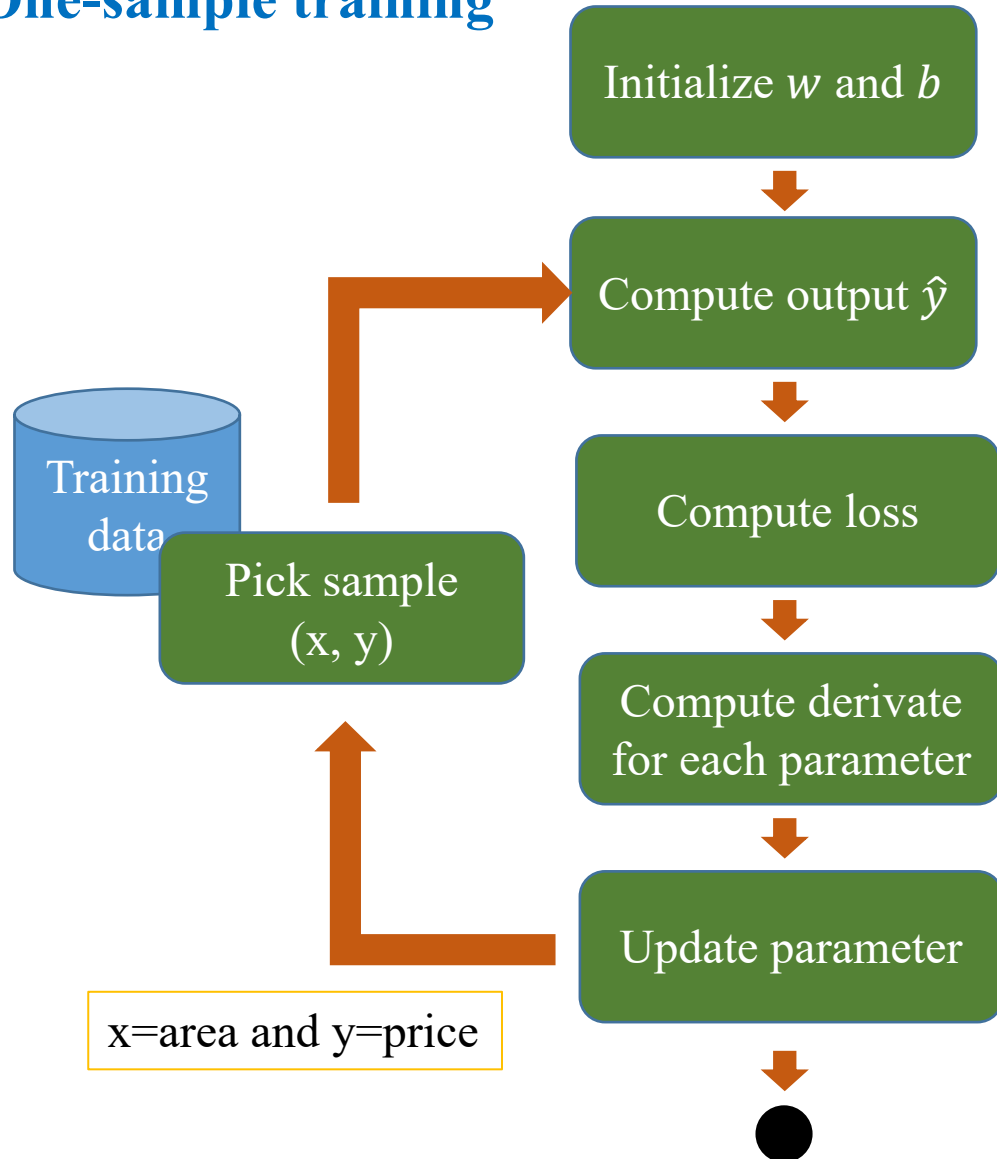
5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

```
1 # forward
2 def predict(x, w, b):
3     return x*w + b
4
5 # compute gradient
6 def gradient(y_hat, y, x):
7     dw = 2*x*(y_hat-y)
8     db = 2*(y_hat-y)
9
10    return (dw, db)
11
12 # update weights
13 def update_weight(w, b, lr, dw, db):
14     w_new = w - lr*dw
15     b_new = b - lr*db
16
17    return (w_new, b_new)
```

Implementation

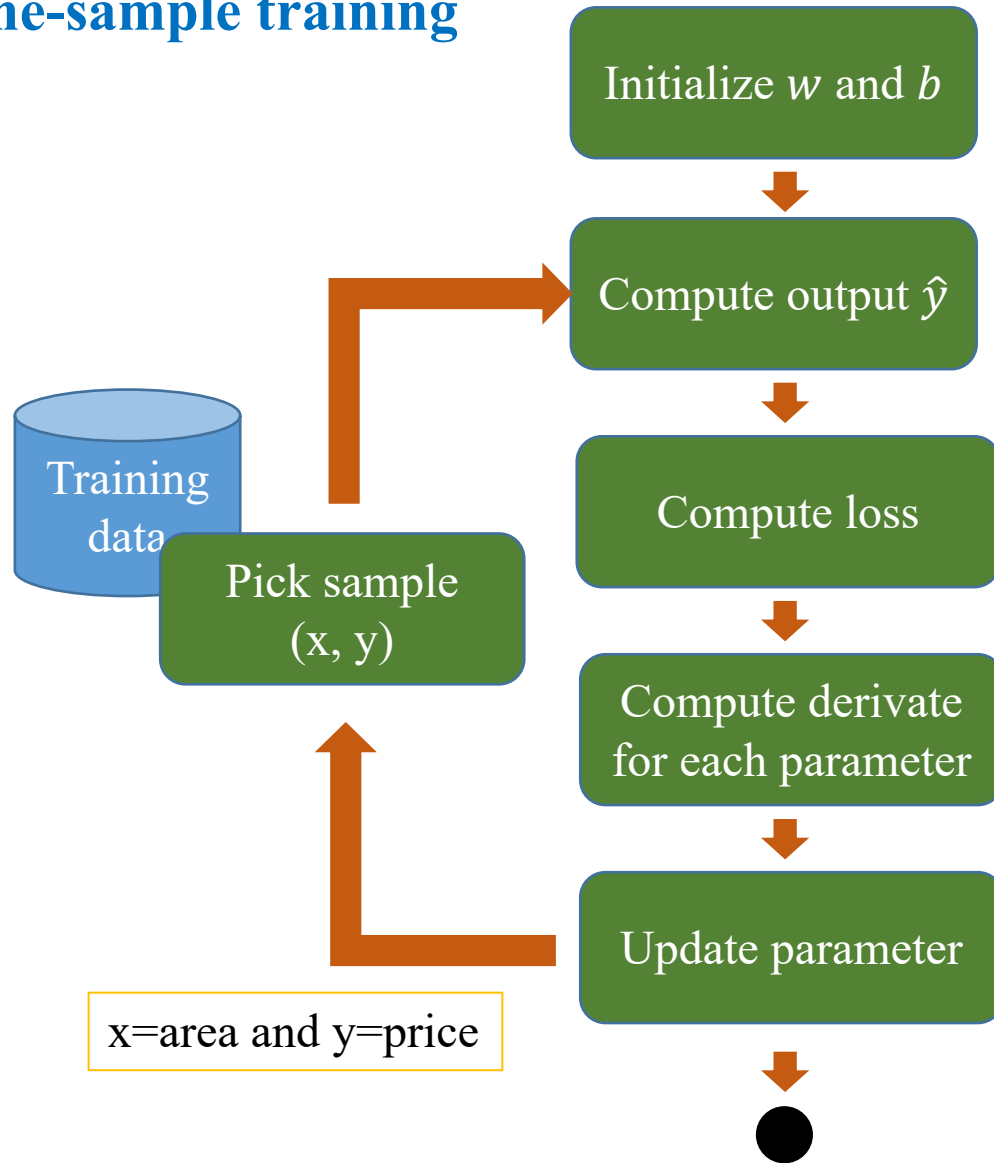
❖ One-sample training



```
1 # init weights
2 b = 0.04
3 w = -0.34
4 lr = 0.01
5
6 # how long
7 epoch_max = 10
8
9 for epoch in range(epoch_max):
10     for i in range(data_size):
11         # get a sample
12         # ...
13
14         # predict y_hat
15         # ...
16
17         # compute loss
18         # ...
19
20         # compute gradient
21         # ...
22
23         # update weights
24         # ...
25
```

Implementation

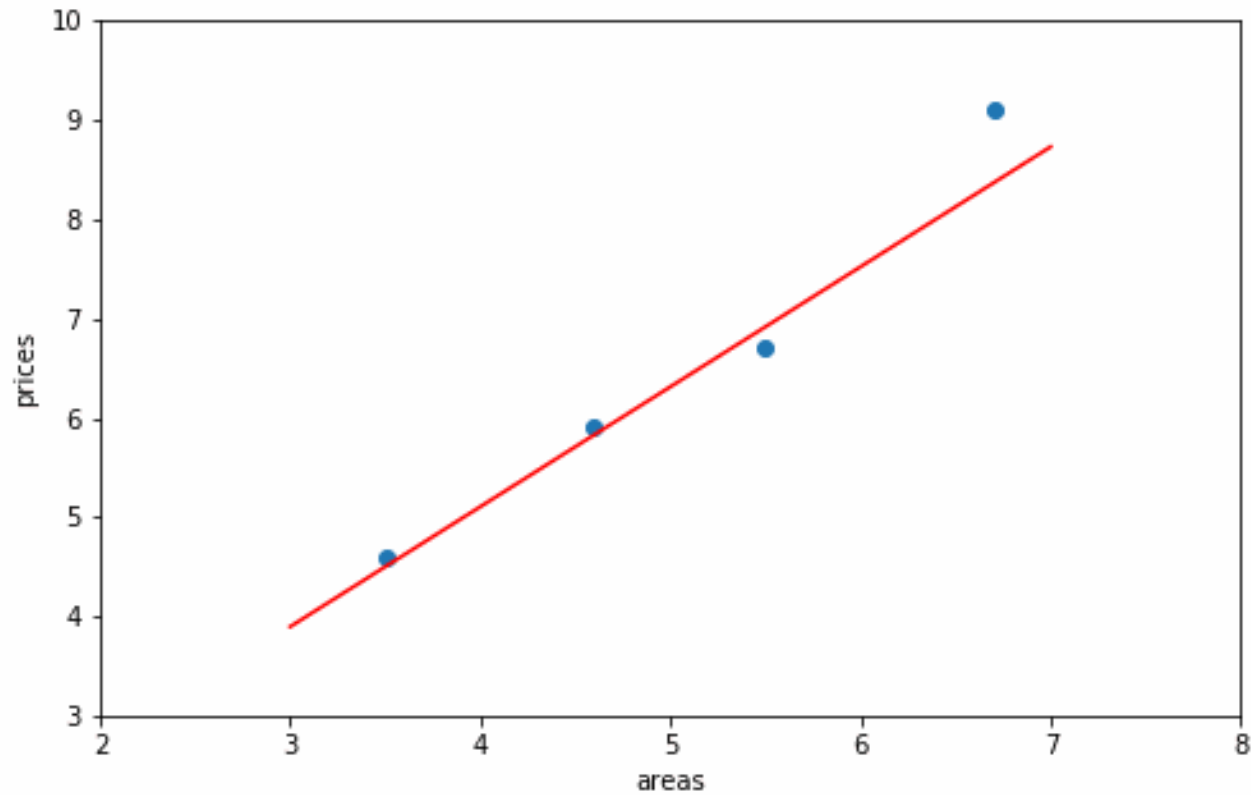
❖ One-sample training



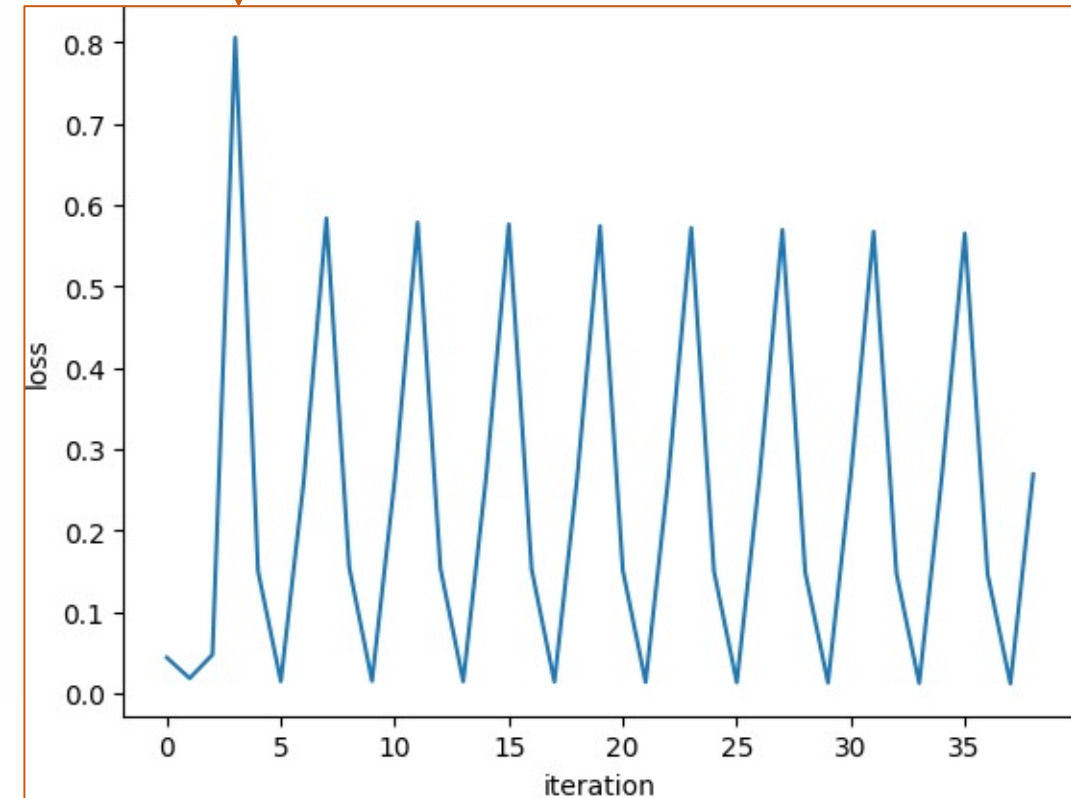
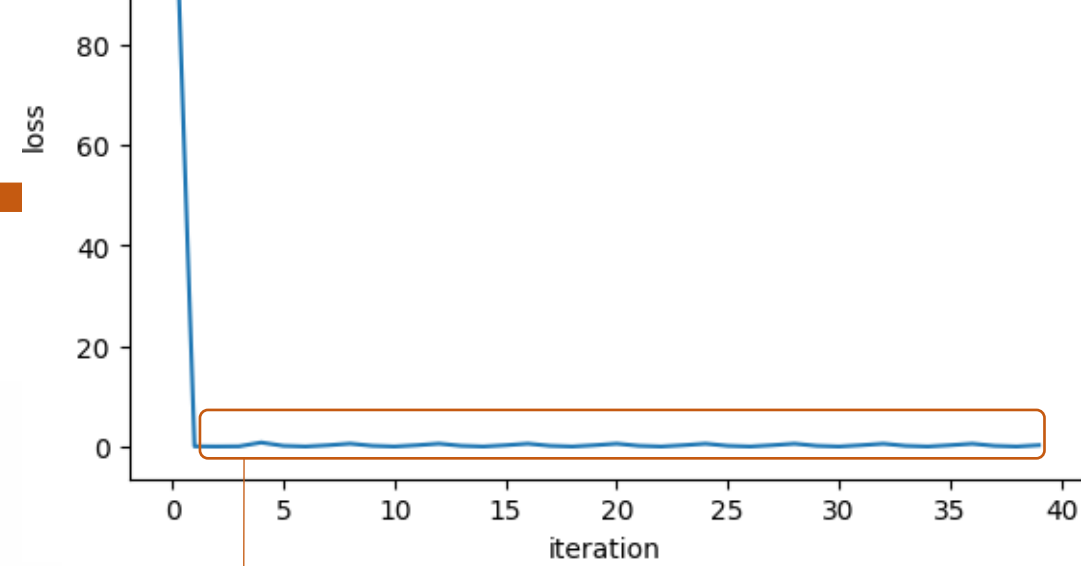
```
1 # init weights
2 b = 0.04
3 w = -0.34
4 lr = 0.01
5
6 # how long
7 epoch_max = 10
8 data_size = 4
9
10 for epoch in range(epoch_max):
11     for i in range(data_size):
12         # get a sample
13         x = areas[i]
14         y = prices[i]
15
16         # predict y_hat
17         y_hat = predict(x, w, b)
18
19         # compute loss
20         loss = (y_hat-y)*(y_hat-y)
21
22         # compute gradient
23         (dw, db) = gradient(y_hat, y, x)
24
25         # update weights
26         (w, b) = update_weight(w, b, lr, dw, db)
```

Implementation

- ❖ House price prediction
 - ❖ One-sample training



Model training



❖ General formula

	Feature	Label	
	area	price	
	6.7	9.1	
	4.6	5.9	
	3.5	4.6	
	5.5	6.7	

House price data

$$\text{Model: } \hat{y} = w_1 x_1 + b$$

$$\widehat{\text{price}} = w_1 * \text{area} + b$$

Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

$$\text{Model: } \hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$\widehat{\text{Sales}} = w_1 * TV + w_2 * Radio + w_3 * Newspaper + b$$

Linear Regression

1) Pick a sample (x_1, x_2, x_3, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = w_1 * TV + w_2 * R + w_3 * N + b$$

$$\hat{y} = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w_1} = 2x_1(\hat{y} - y) \quad \frac{\partial L}{\partial w_3} = 2x_3(\hat{y} - y)$$

$$\frac{\partial L}{\partial w_2} = 2x_2(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} \quad w_3 = w_3 - \eta \frac{\partial L}{\partial w_3}$$

$$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2} \quad b = b - \eta \frac{\partial L}{\partial b}$$

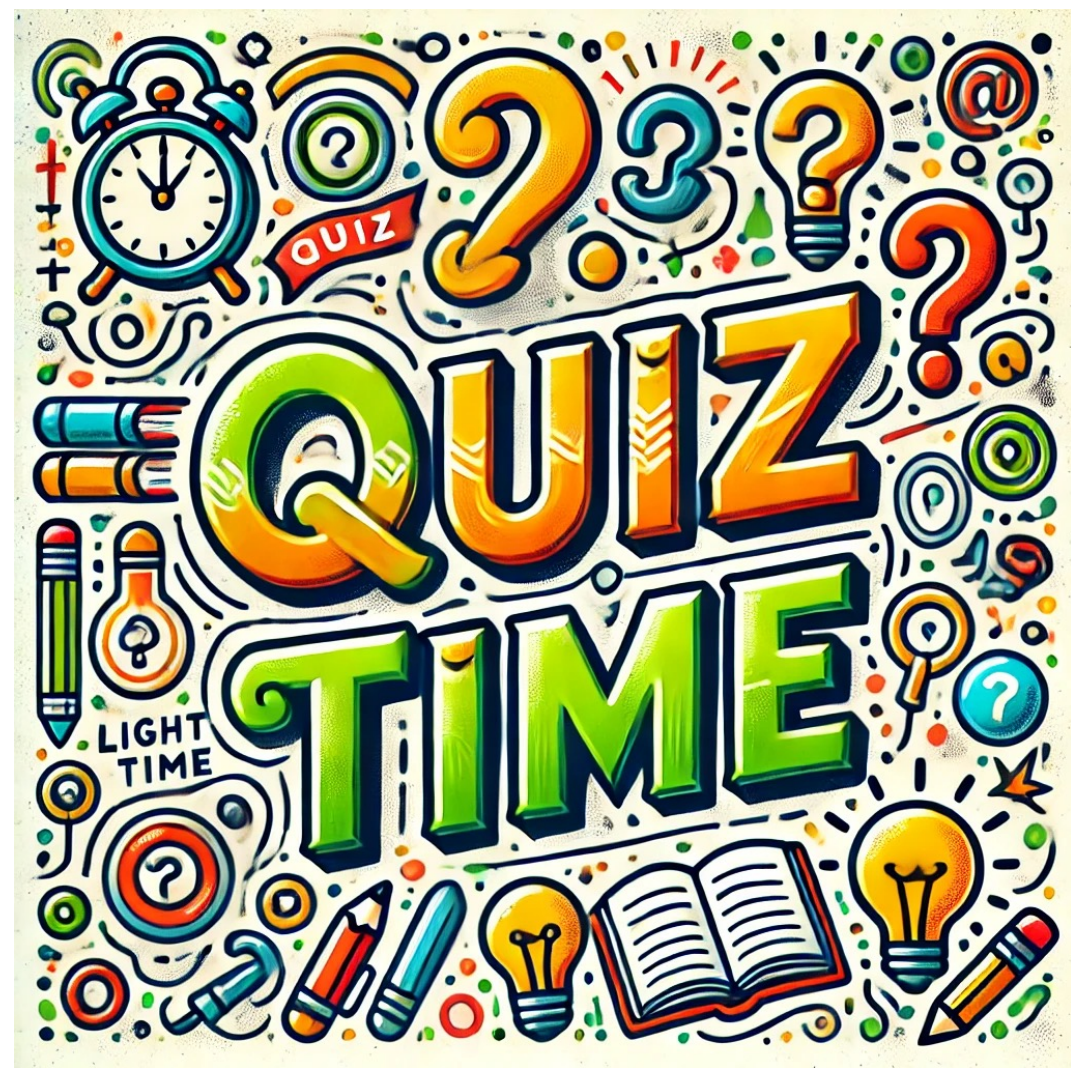
Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

Model

$$\text{Sale} = w_1 * TV + w_2 * Radio + w_3 * Newspaper + b$$

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$



Outline

SECTION 1

Gradient descent

SECTION 2

Linear Regression

SECTION 3

Vectorization

$$\begin{cases} b = b - \eta \frac{\partial L}{\partial b} \\ w = w - \eta \frac{\partial L}{\partial w} \\ \theta = \theta - \eta \nabla_{\theta} L \end{cases}$$

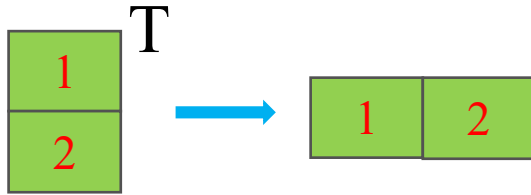
$$\rightarrow \theta = \theta - \eta \nabla_{\theta} L$$

Linear Regression (1-samples)

Transpose

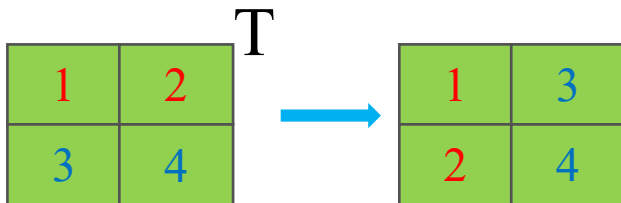
$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix}$$

$$\vec{v}^T = [v_1 \ \dots \ v_n]$$



$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix}$$

$$A^T = \begin{bmatrix} a_{11} & \dots & a_{m1} \\ \dots & \dots & \dots \\ a_{1n} & \dots & a_{mn} \end{bmatrix}$$

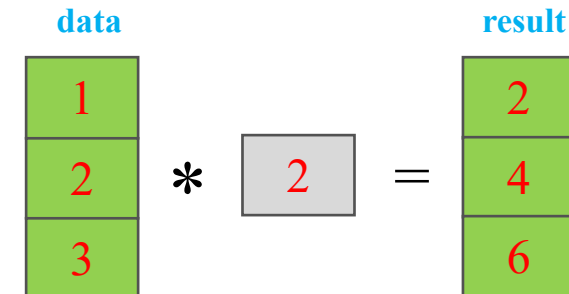


```
2 import numpy as np
3
4 # create data
5 data = np.array([1,2,3])
6 factor = 2
7
8 # broadcasting
9 result_multiplication = data*factor
```

```
[1 2 3]
[2 4 6]
```

Multiply with a number

$$\alpha \vec{u} = \alpha \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} \alpha u_1 \\ \dots \\ \alpha u_n \end{bmatrix}$$



Linear Regression (1-samples)

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Traditional

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\theta = \begin{bmatrix} b \\ w \end{bmatrix} \rightarrow \theta^T = [b \quad w]$$

$$\hat{y} = wx + b1 = [b \quad w] \begin{bmatrix} 1 \\ x \end{bmatrix} = \theta^T x$$

dot product

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Traditional

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\hat{y} = \theta^T x$$

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

numbers

What will we do?

Linear Regression (1-samples)

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

Traditional

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\left\{ \begin{array}{l} \frac{\partial L}{\partial b} = 2(\hat{y} - y) = 2 \times (\hat{y} - y) \times 1 \\ \frac{\partial L}{\partial w} = 2x(\hat{y} - y) = 2 \times (\hat{y} - y) \times x \end{array} \right.$$

$$\begin{bmatrix} 2 \times (\hat{y} - y) \times 1 \\ 2 \times (\hat{y} - y) \times x \end{bmatrix} = \underbrace{2(\hat{y} - y)}_{\text{common factor}} \begin{bmatrix} 1 \\ x \end{bmatrix} = 2(\hat{y} - y)x = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix} = \nabla_{\theta} L \quad \rightarrow \quad \nabla_{\theta} L = 2x(\hat{y} - y)$$

Linear Regression (1-samples)

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

Traditional

$$\hat{y} = wx + b$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ x \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\nabla_{\boldsymbol{\theta}} L = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix}$$

$$\left\{ \begin{array}{l} b = b - \eta \frac{\partial L}{\partial b} \\ w = w - \eta \frac{\partial L}{\partial w} \end{array} \right. \quad \nabla_{\boldsymbol{\theta}} L$$

$$\rightarrow \boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Traditional

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \mathbf{x} = \mathbf{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = 2\mathbf{x}(\hat{y} - y)$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate

Vectorized

$$\hat{y} = wx + b$$

$$x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

1

$$x = \begin{bmatrix} 1 \\ x \end{bmatrix} = \begin{bmatrix} 1 \\ 6.7 \end{bmatrix}$$

Given $\theta = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.049 \\ -0.34 \end{bmatrix}$

$$\eta = 0.01$$

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \theta^T x = x^T \theta$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

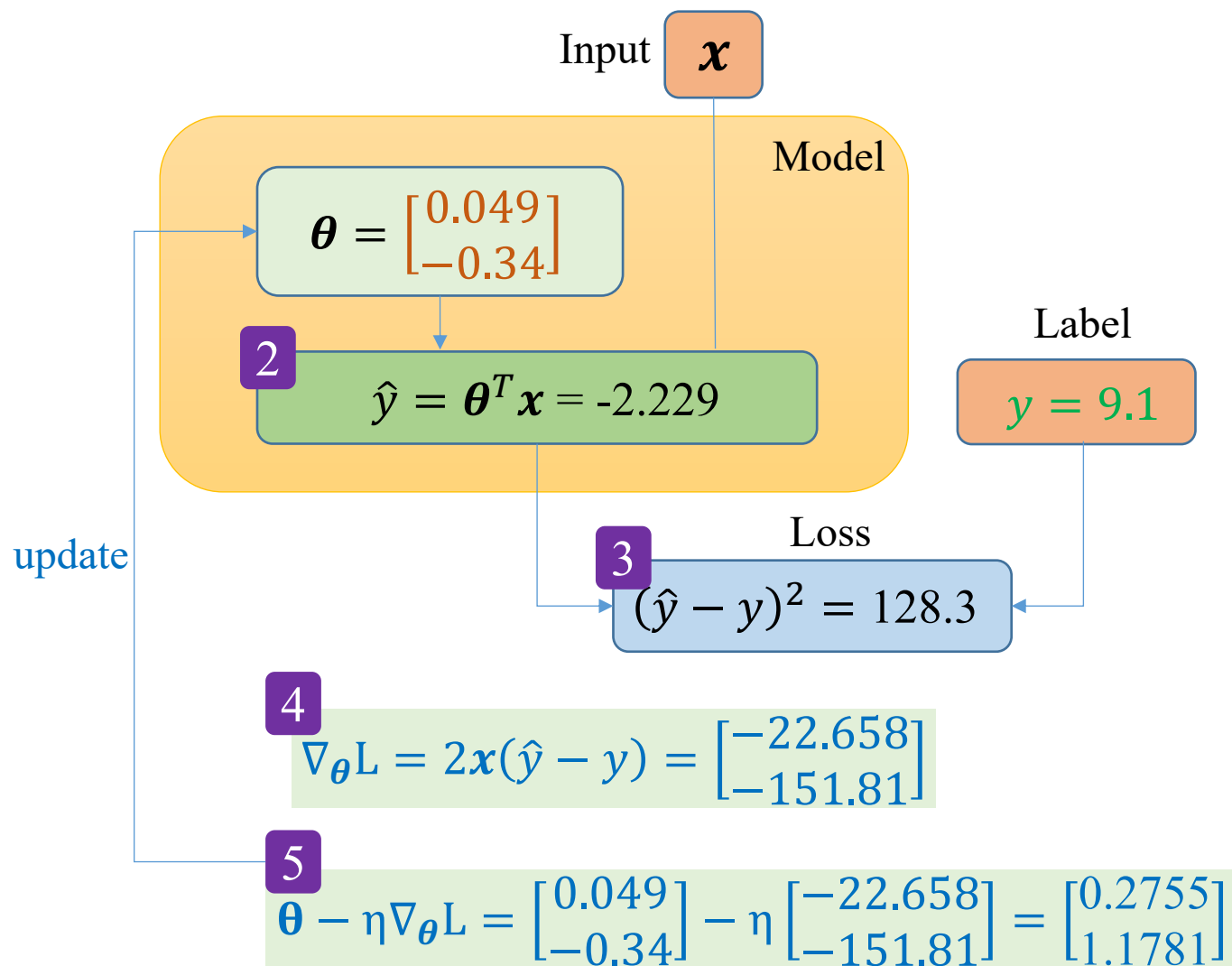
4) Compute derivative

$$\nabla_{\theta} L = 2x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate



❖ Implementation (vectorization using numpy)

demo

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \theta^T x = x^T \theta$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\theta} L = 2x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

```
1 import numpy as np
2
3 # forward
4 def predict(x, theta):
5     return x.dot(theta)
6
7 # compute gradient
8 def gradient(y_hat, y, x):
9     dtheta = 2*x*(y_hat-y)
10
11     return dtheta
12
13 # update weights
14 def update_weight(theta, lr, dtheta):
15     dtheta_new = theta - lr*dtheta
16
17     return dtheta_new
```

$$\hat{y} = wx + b$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

1

$$\mathbf{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$$

Given $\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$

$$\eta = 0.01$$

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \mathbf{x} = \mathbf{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

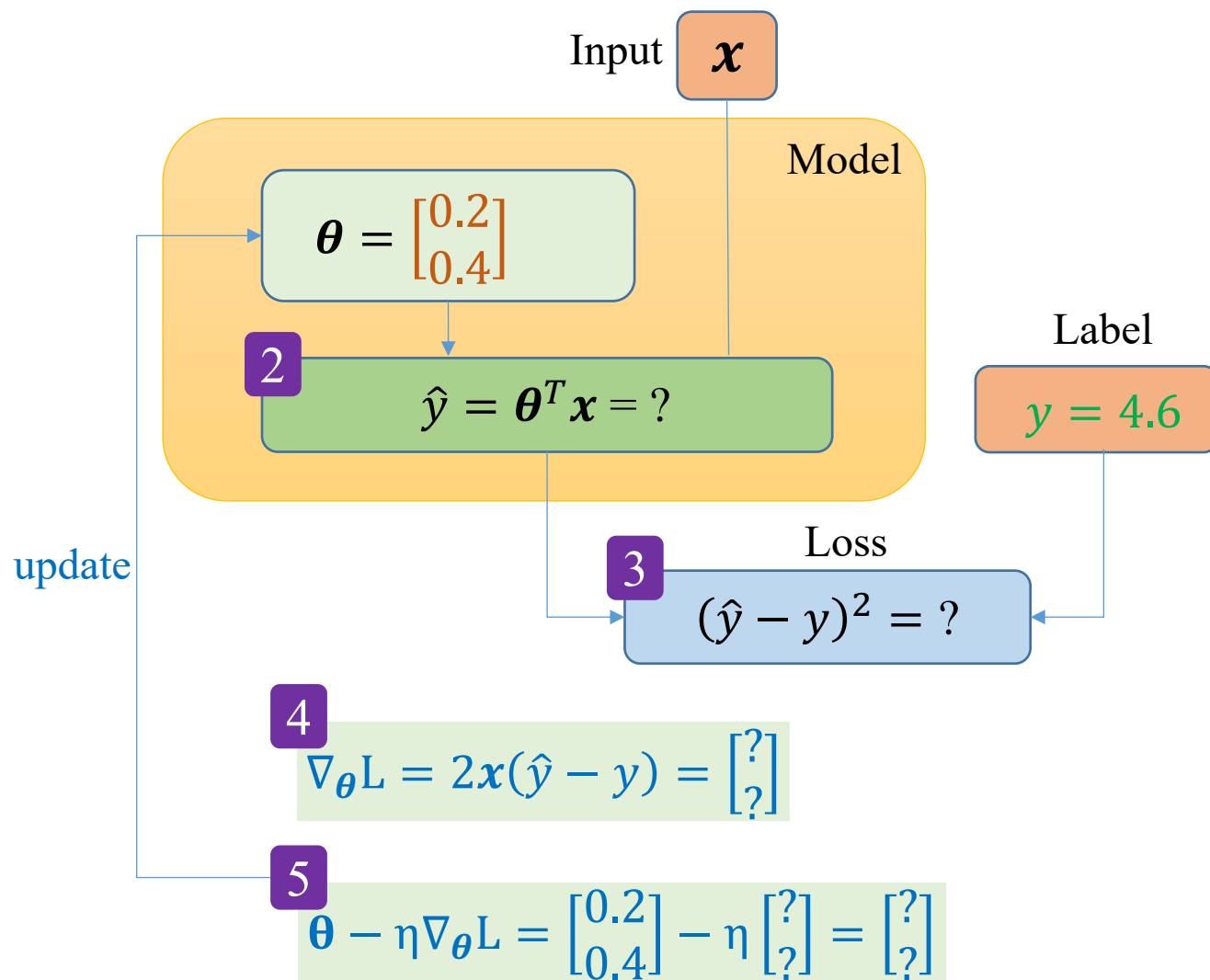
4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} L = 2\mathbf{x}(\hat{y} - y)$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

η is learning rate



```

3 data = np.genfromtxt('data.csv', delimiter=',')
4 N = 4
5
6 areas = data[:, 0].reshape(N, 1)
7 prices = data[:, 1].reshape(N,)
8
9 # vector [1, area]
10 features = np.hstack([np.ones((N,1)), areas])

```

```

1 # forward
2 def predict(x, theta):
3     return x.T.dot(theta)
4
5 # compute gradient
6 def gradient(y_hat, y, x):
7     dtheta = 2*x*(y_hat-y)
8     return dtheta
9
10 # update weights
11 def update_weight(theta, lr, dtheta):
12     dtheta_new = theta - lr*dtheta
13     return dtheta_new

```

```

1 lr = 0.01
2 epoch_max = 10
3
4 # [b, w]
5 theta = np.array([0.049, -0.34])
6
7 for epoch in range(epoch_max):
8     for i in range(N):
9         # get a sample
10         x = features[i,:]
11         y = prices[i]
12
13         # predict y_hat
14         y_hat = predict(x, theta)
15
16         # compute loss
17         loss = (y_hat-y)*(y_hat-y)
18
19         # compute gradient
20         dtheta = gradient(y_hat, y, x)
21
22         # update weights
23         theta = update_weight(theta, lr, dtheta)

```

Advertising Problem

Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

if TV=55.0, Radio=34.0,
and Newspaper=62.0,
price=?

$$\hat{y} = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$$

$$x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\theta = \begin{bmatrix} b \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

1) Pick a sample (x, y) from training data



2) Compute output $\hat{y}$



$$\hat{y} = \theta^T x = x^T \theta$$

3) Compute loss



$$L = (\hat{y} - y)^2$$

4) Compute derivative



$$\nabla_{\theta} L = 2x(\hat{y} - y)$$

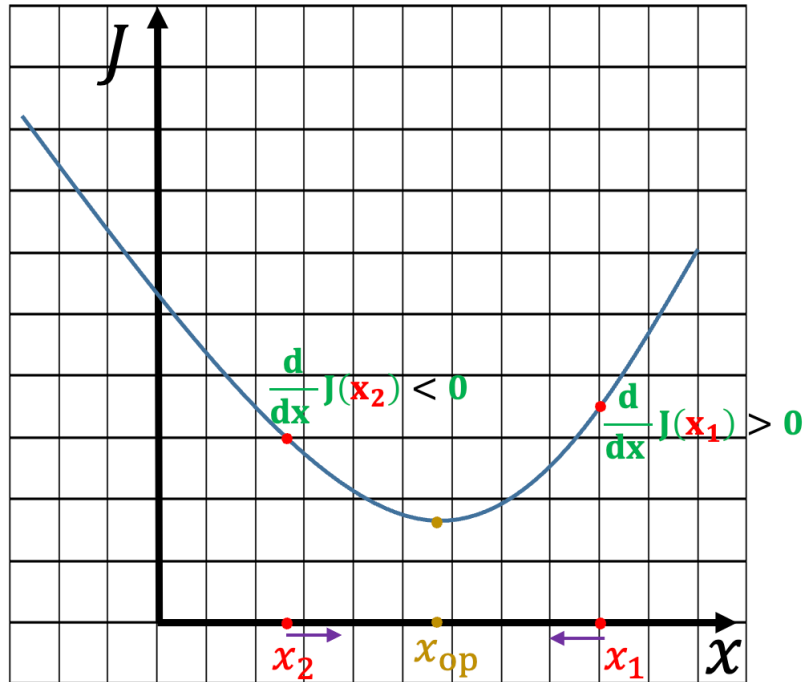
5) Update parameters

$$\theta = \theta - \eta \nabla_{\theta} L$$

η is learning rate

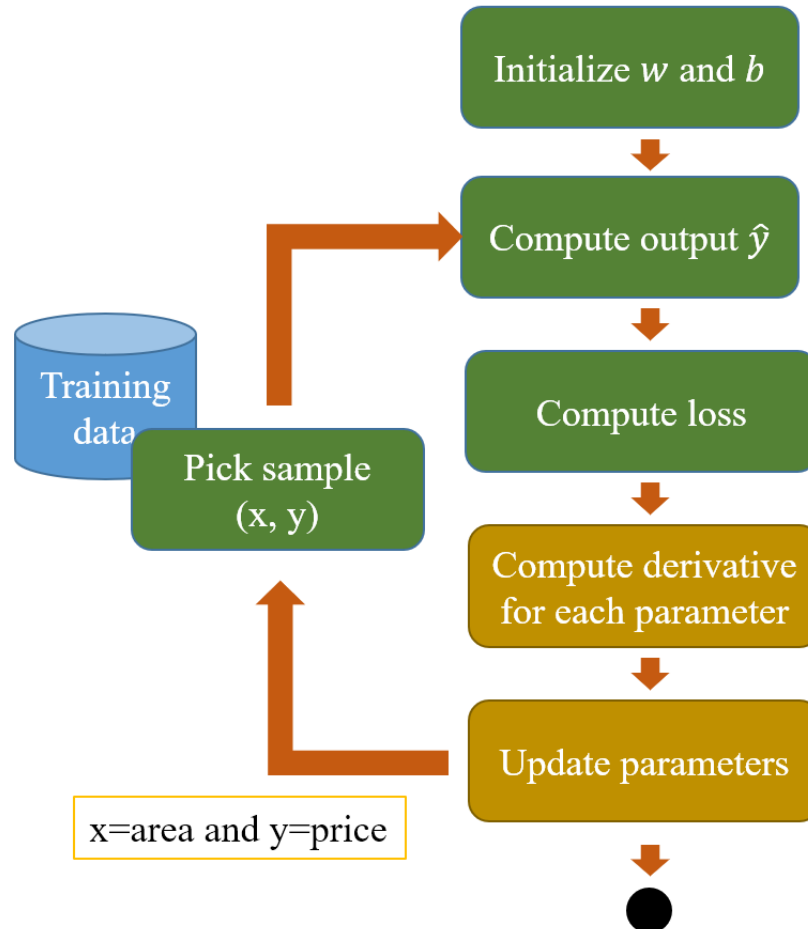
Summary

Gradient descent



$$\mathbf{x}_{new} = \mathbf{x}_{old} - \eta \frac{d}{dx}J(\mathbf{x}_{old})$$

Linear Regression



Vectorization

$$\begin{cases} b = b - \eta \frac{\partial L}{\partial b} \\ w = w - \eta \frac{\partial L}{\partial w} \\ \theta \end{cases} \quad \nabla_{\theta} L$$

$$\rightarrow \theta = \theta - \eta \nabla_{\theta} L$$

